

Horarios 2025

Turno 1)

- Teoría: Lunes de 16:00 a 18:30
- Práctica: Viernes de 16:30 a 18:30

Turno 2)

- Teoría: Miércoles de 18:00 a 20:30
- Práctica: Viernes de 18:30 a 20:30

Plataforma Ideas

<https://ideas.info.unlp.edu.ar>

Seminario de Lenguajes - Opción .NET 1er. Semestre 2025

Usaremos la **Cartelera**, la **Mensajería** y el **Foro**,

En **Cartelera de novedades** hay un link a una carpeta de Google Drive donde iremos agregando el material del curso (ya está disponible la **Propuesta de enseñanza** en dónde puede consultarse la **metodología** y el **cronograma** completo de las actividades del curso).

Método de Evaluación

Aprobación de la cursada

- Cumplir con la entrega parcial y aprobar la entrega final de un trabajo integrador con nota mayor o igual a 6 (seis). Este trabajo se puede presentar de manera individual o grupal. En caso de desaprobado habrá una fecha más de re-entrega.
- Aprobar un coloquio final e individual.

Promoción

- El alumno que aprueba la cursada obtiene la promoción (La nota contemplará las entregas y el coloquio final).



Descargar e instalar la versión .NET 8.0

<https://dotnet.microsoft.com/download/dotnet>

^ Versiones admitidas

Versión	Tipo de versión	Fase de soporte técnico	Última versión	Fecha de lanzamiento más reciente	Finalización del soporte
.NET 10.0	Compatibilidad a largo plazo ⓘ	Vista previa ⓘ	10.0.0-preview.1	25 de febrero de 2025	
.NET 9.0 (última)	Soporte técnico de términos estándar ⓘ	Activo ⓘ	9.0.2	11 de febrero de 2025	12 de mayo de 2026
.NET 8.0	Compatibilidad a largo plazo ⓘ	Activo ⓘ	8.0.13	11 de febrero de 2025	10 de noviembre de 2026





Descargar e instalar la versión .NET 8.0

Vamos a trabajar **.NET 8.0** (soportada hasta el **10/11/2026**)

La versión **.NET 9.0** no ha traído mejoras relevantes y es de soporte corto (soportada hasta **12/05/2026**)

^ Versión						
Versión	Tiempo					Finalización del soporte
.NET 10.0	preview.1			2025		
.NET 9.0 (última)	Soporte técnico de términos estándar	Activo	9.0.2	11 de febrero de 2025	12 de mayo de 2026	
.NET 8.0	Compatibilidad a largo plazo	Activo	8.0.13	11 de febrero de 2025	10 de noviembre de 2026	



Para este curso vamos a necesitar descargar e instalar ...



Descargar e instalar Visual Studio Code (<https://code.visualstudio.com/>)

 Visual Studio Code Docs Updates Blog API Extensions FAQ [Download](#)

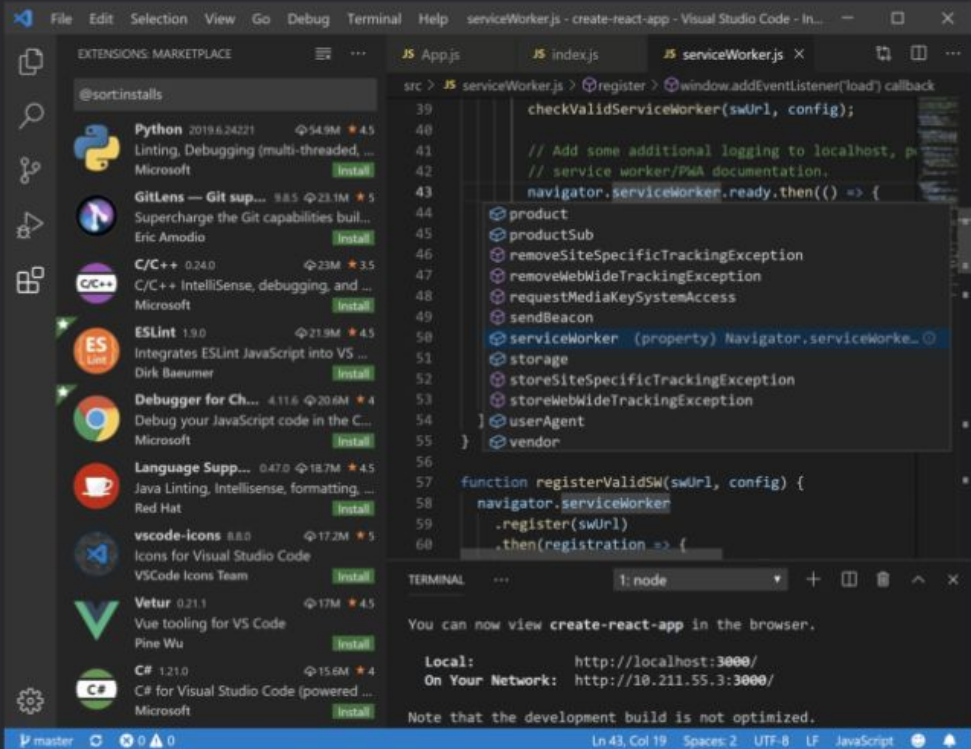
Version 1.48 is now available! Read about the new features and fixes from July.

Code editing. Redefined.

Free. Built on open source. Runs everywhere.

[Download for Windows](#)
Stable Build

		Stable	Insiders
macOS	Package	↓	↓
Windows x64	User Installer	↓	↓
Linux x64	.deb .rpm	↓ ↓	↓ ↓
Other downloads			





.NET

Teoría 1

Generalidades de .NET



Un poco de historia .NET Framework



- La versión 1.0 de .NET Framework fue lanzada oficialmente por Microsoft para su sistema operativo Windows en enero del 2002.
- Junto con .NET Framework, Microsoft liberó la primera versión del lenguaje C# y una nueva versión de Visual Basic.
- F# es un lenguaje funcional más reciente, fue desarrollado por Microsoft en 2005.
- .NET Framework es gratuito pero no es *open source*. La última versión es la 4.8 (última actualización abril de 2019).

Un poco de historia .NET Core y .Net



- La versión **multiplataforma** (Windows, macOS y Linux) y **open-source** de **.NET** se inició con el nombre de **.NET Core** (v 1.0 liberado en junio 2016)
- La última versión con el nombre **.NET Core** es la **3.1** (liberado en diciembre de 2019). A partir de noviembre de 2020 cambió su nombre a **.NET 5.0**
- En noviembre de 2021 se liberó **NET 6.0**.
- En noviembre de 2022 se liberó **NET 7.0**.
- En noviembre de 2023 se liberó **NET 8.0**.

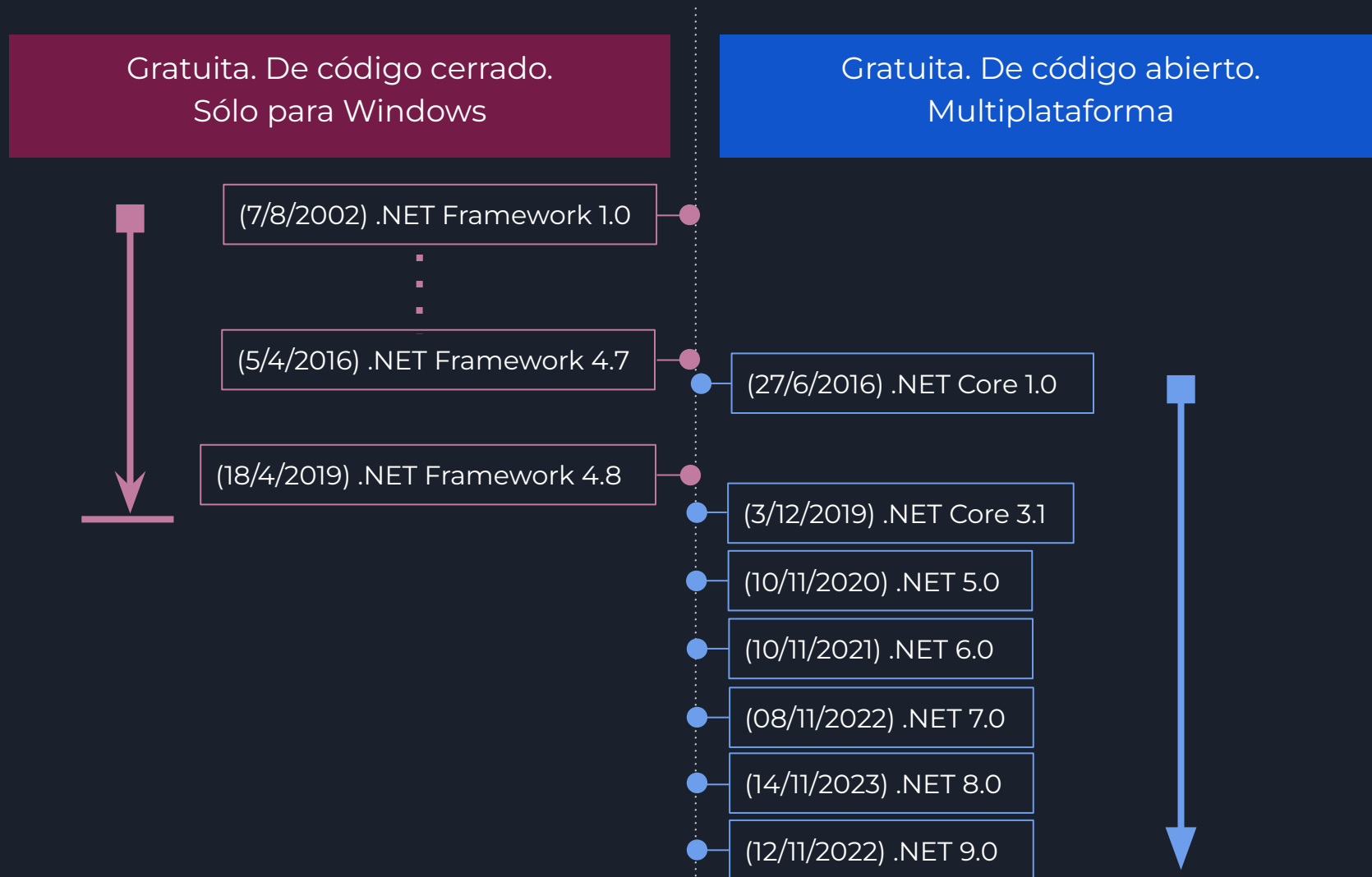


.NET Core 3.1 evolucionó a .NET 5.0

.NET 5.0 NO es la continuación de .NET Framework 4.8 sino la continuación de .NET Core 3.1

Aunque pudo llamarse .NET Core 5.0, se prefirió cambiar el nombre a .NET 5.0 para dar idea de unificación. En el futuro sólo se hablará de .NET

Generalidades de .NET - Evolución

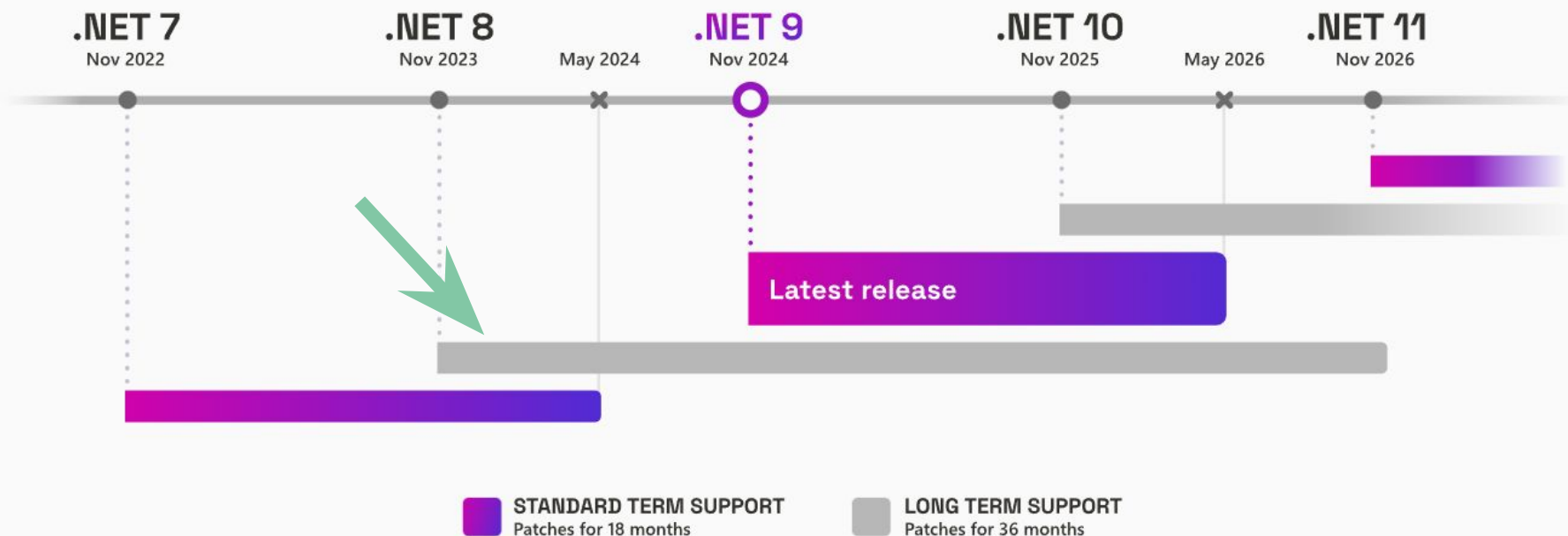


Hoja de ruta de .NET

(<https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core>)

Cadencia de lanzamiento de .NET

Las versiones numeradas impares son versiones STS que obtienen soporte técnico gratuito y revisiones durante 18 meses.



Generalidades de .NET

- **.NET** es una plataforma de desarrollo gratuita, de código abierto y de uso general
- Soporta varios lenguajes de programación. **Microsoft** desarrolla activamente **C#**, **Visual Basic** y **F#**, pero también existen otros.

Tipos de aplicaciones que se pueden construir con .NET



Web

Build web apps and services for Linux, Windows, macOS, and Docker.



Mobile

Use a single codebase to build native mobile apps for iOS, Android, and Windows.



Desktop

Create beautiful and compelling desktop apps for Windows and macOS.



Microservices

Create independently deployable microservices that run on Docker containers.



Cloud

Consume existing cloud services, or create and deploy your own.



Machine Learning

Add vision algorithms, speech processing, predictive models, and more to your apps.



Game Development

Develop 2D and 3D games for the most popular desktops, phones, and consoles.



Internet of Things

Make IoT apps, with native support for the Raspberry Pi and other single-board computers.



Common Language Runtime (CLR)

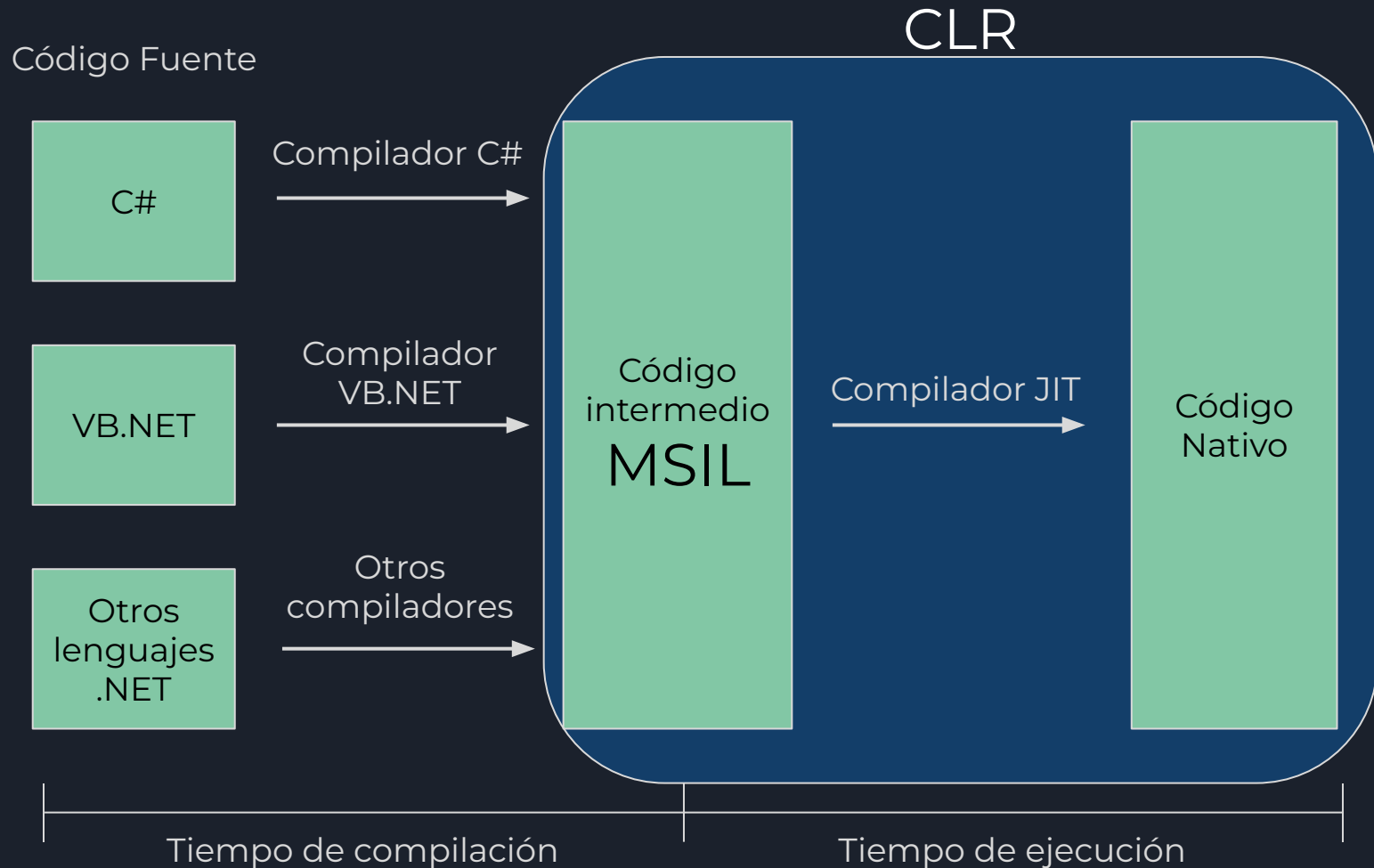
- El CLR es el motor de ejecución (*runtime*) de .NET
- Es un entorno virtual independiente de la arquitectura de hardware en el que se ejecutan las aplicaciones escritas con cualquier lenguaje .NET

Microsoft Intermediate Language (MSIL)



- Cuando se compila una aplicación escrita en un lenguaje de .NET (VB, C# u otro de los soportados), el compilador genera código MSIL (también conocido como código CIL).
- MSIL es un lenguaje similar a un código ensamblador pero de más alto nivel, creado para un hipotético procesador virtual que no está atado a una arquitectura determinada.
- De manera transparente el código MSIL se traduce al lenguaje nativo del procesador físico en tiempo de ejecución, por intermediación de un compilador bajo demanda “Just In Time” (JIT)

Microsoft Intermediate Language (MSIL)



Base Classes Library (BCL)

- La **BCL** es la biblioteca de clases base de **.NET** que da soporte a infinidad de funcionalidades a través de las **clases** y otros **tipos** definidos en ella.
- Las clases en la **BCL** se organizan en espacios de nombres (**namespaces**) que agrupan a clases y a otros **namespaces**.
- A modo de ejemplo:
 - Los **tipos integrados** que veremos más adelante en esta teoría están en el espacio de nombres **System**
 - En **System.Collections** (namespace **Collections** dentro del namespace **System**) encontramos clases que representan colecciones
 - **System.IO** proporciona clases para trabajar con operaciones de entrada y salida, lo que incluye operaciones de archivos, directorios y flujos de datos



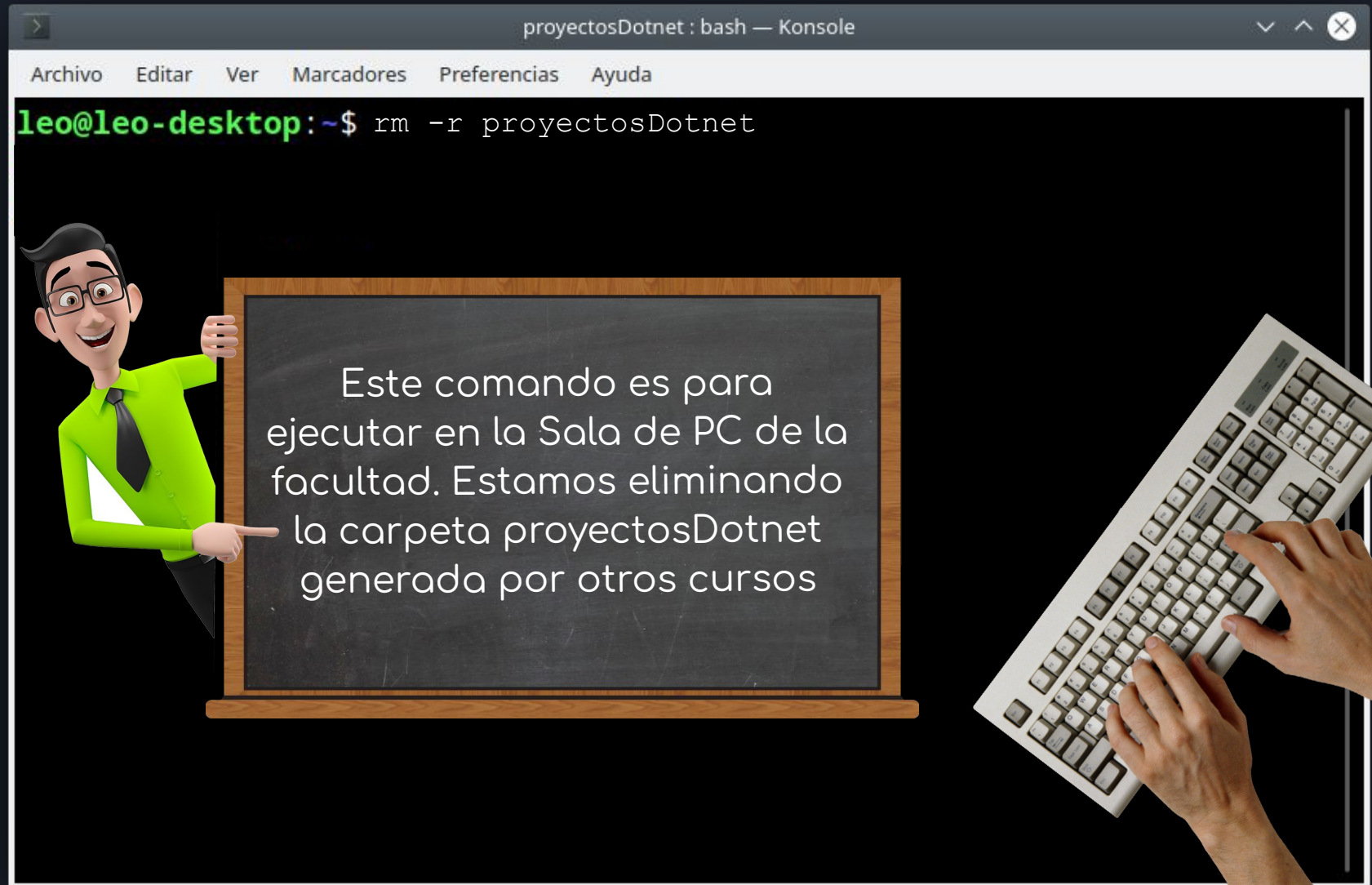
Algunos namespaces de la BCL

Nombre	Descripción
System	El espacio de nombres contiene clases fundamentales y clases base que definen tipos de datos de valor y de referencia usados comúnmente, eventos y controladores de eventos, interfaces, atributos y excepciones de procesamiento.
System.Web	El espacio de nombres proporciona clases e interfaces que permiten la comunicación entre el explorador y el servidor.
System.Media	El espacio de nombres contiene clases para reproducir archivos de sonido y obtener acceso a los sonidos que proporciona el sistema.
System.Printing	Proporciona clases que permiten automatizar la administración de servidores, colas y trabajos de impresión.
System.Xml	El espacio de nombres proporciona compatibilidad basada en estándares para procesar XML.
System.Data	El espacio de nombres proporciona acceso a las clases que representan la arquitectura de ADO.NET. ADO.NET permite crear componentes que administran datos de varios orígenes de datos con eficacia.
System.Collections	El espacio de nombres contiene interfaces y clases que definen varias colecciones de objetos, como listas, colas, matrices de bits, tablas hash y diccionarios.
System.Text	El espacio de nombres contiene clases que representan codificaciones de caracteres ASCII y Unicode; clases base abstractas para convertir bloques de caracteres a y desde bloques de bytes y una clase del asistente que manipula y da formato a objetos sin crear instancias intermedias de .

Introducción a C#



Abrir una terminal (consola) del sistema operativo y tipear ...





Abrir una terminal (consola) del sistema operativo y tipear ...



leo : bash — Konsole

Archivo Editar Ver Marcadores Preferencias Ayuda

leo@leo-desktop: ~\$ `mkdir proyectosDotnet`

Crear el directorio
proyectosDotnet

NOTA: Los términos carpeta y directorio son sinónimos en este contexto



Abrir una terminal (consola) del sistema operativo y tipear ...

```
leo : bash — Konsole
Archivo  Editar  Ver  Marcadores  Preferencias  Ayuda
leo@leo-desktop:~$ mkdir proyectosDotnet
leo@leo-desktop:~$ cd proyectosDotnet
```

Cambiar al directorio
proyectosDotnet

Command-line interface (CLI)

Para crear nuestra aplicación vamos a utilizar
.NET CLI

La interfaz de la línea de comandos de **.NET** es un conjunto de herramientas multiplataforma que sirve para desarrollar, compilar, ejecutar y publicar aplicaciones **.NET**.

Todos los comandos comienzan con **dotnet** que es el controlador genérico de la **CLI** de **.NET**.





Abrir una terminal (consola) del sistema operativo y tipear ...

```
leo@leo-desktop:~$ mkdir proyectosDotnet
leo@leo-desktop:~$ cd proyectosDotnet
leo@leo-desktop:~/proyectosDotnet$ dotnet new console -o teoria1
```

Crear la aplicación de consola `teoria1`

Comando dotnet

```
leo@leo-desktop:~$ mkdir proyectosDotnet
leo@leo-desktop:~$ cd proyectosDotnet
leo@leo-desktop:~/proyectosDotnet$ dotnet new console -o teoria1
```

NOTA: Para ver los distintos tipos de proyectos que se pueden crear se debe tipear:
`dotnet new list`
o simplemente
`dotnet new` para ver los más comunes

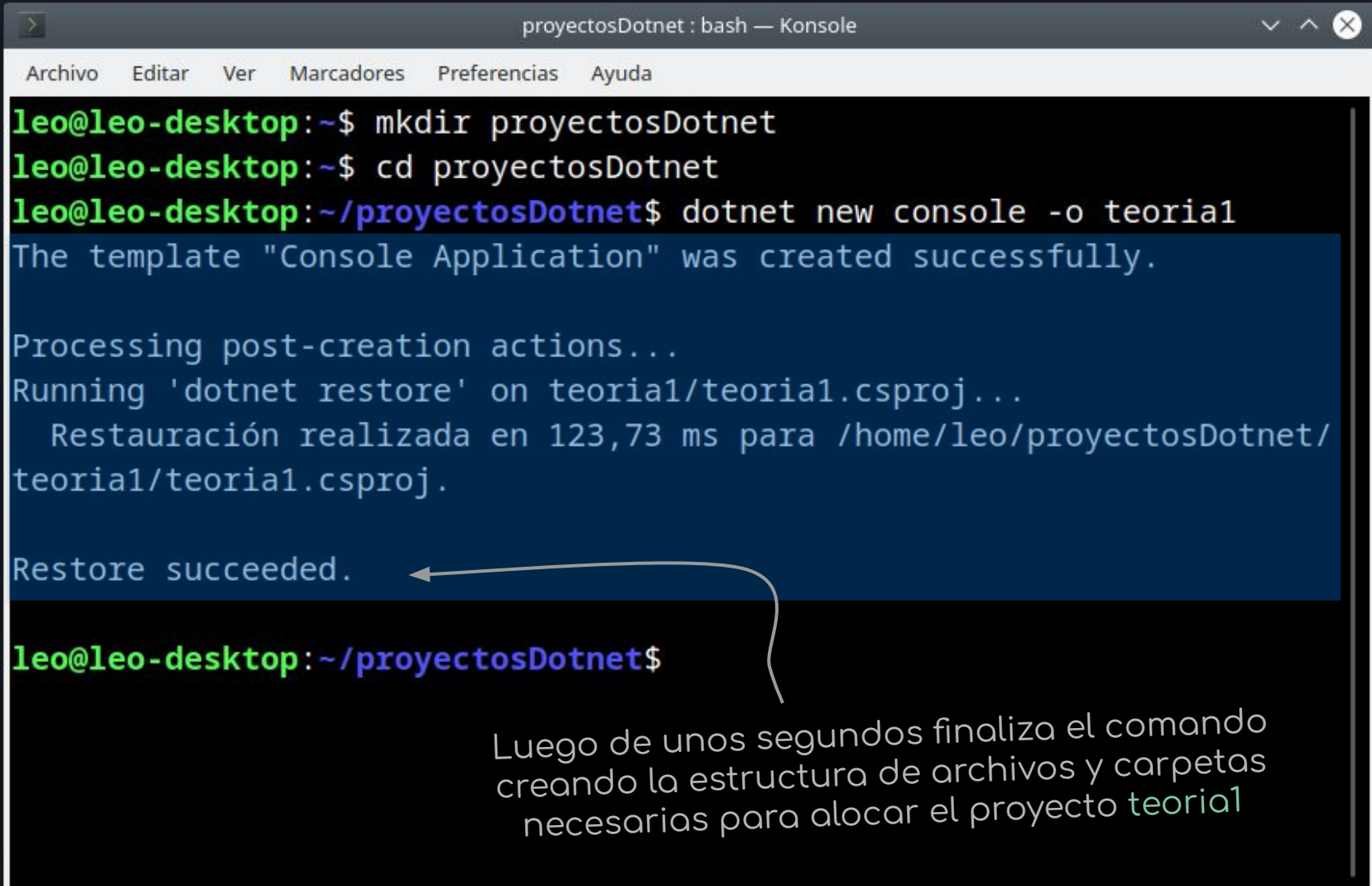
comando para crear un proyecto basado en una plantilla

Nombre de la plantilla para crear una aplicación de consola

`-o (output)`
establece la carpeta (se crea si es necesario) donde se genera el proyecto



Comando dotnet



```
proyectosDotnet : bash — Konsole
Archivo  Editar  Ver  Marcadores  Preferencias  Ayuda
leo@leo-desktop:~$ mkdir proyectosDotnet
leo@leo-desktop:~$ cd proyectosDotnet
leo@leo-desktop:~/proyectosDotnet$ dotnet new console -o teoria1
The template "Console Application" was created successfully.

Processing post-creation actions...
Running 'dotnet restore' on teoria1/teoria1.csproj...
  Restauración realizada en 123,73 ms para /home/leo/proyectosDotnet/
teoria1/teoria1.csproj.

Restore succeeded.
```

Luego de unos segundos finaliza el comando creando la estructura de archivos y carpetas necesarias para alojar el proyecto `teoria1`



En la terminal del sistema operativo hacer:

```
Processing post-creation actions...  
Running 'dotnet restore' on teoria1/teoria1.csproj...  
  Restauración realizada en 123,73 ms para /home/leo/proyectosDotnet/teoria1/teoria1.csproj.
```

```
Restore succeeded.
```

```
leo@leo-desktop:~/proyectosDotnet$ cd teoria1
```

```
leo@leo-desktop:~/proyectosDotnet/teoria1$ ls
```

Cambiar al directorio **teoria1** ...

... y listar los archivos generados

En Windows (Símbolo del Sistema) se puede usar el comando **dir** en lugar de **ls**
Windows PowerShell soporta el comando **ls**

Contenido del proyecto generado por el comando dotnet new

```
Processing post-creation actions...
Running 'dotnet restore' on teoria1/teoria1.csproj...
  Restauración realizada en 123,73 ms para /home/leo/proyectosDotnet/
teoria1/teoria1.csproj.

Restore succeeded.

leo@leo-desktop:~/proyectosDotnet$ cd teoria1
leo@leo-desktop:~/proyectosDotnet/teoria1$ ls
obj  Program.cs  teoria1.csproj
leo@leo-desktop:~/proyectosDotnet/teoria1$
```

En el archivo
Program.cs hay
código C Sharp

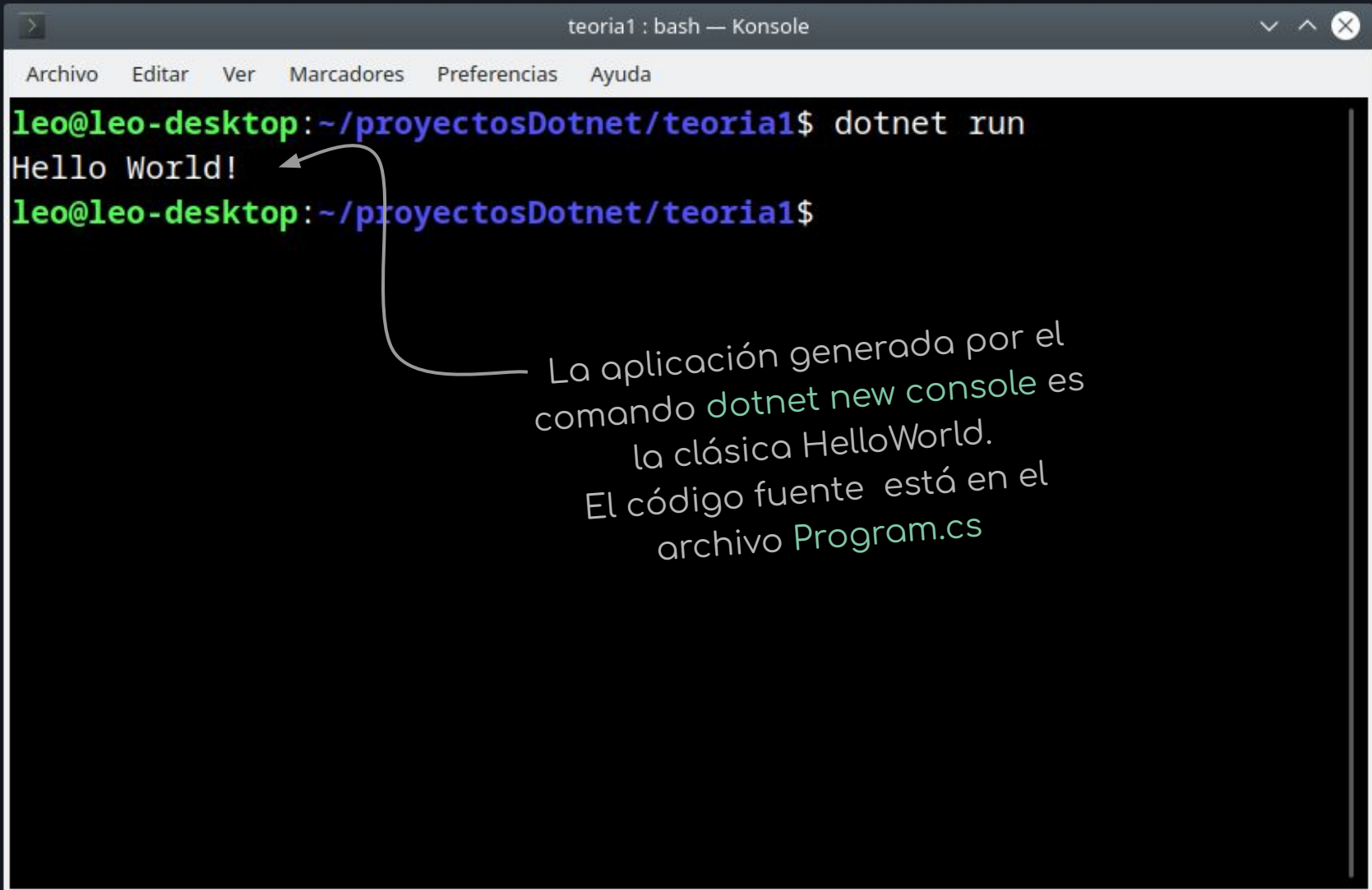


En la terminal del sistema operativo hacer:

```
teoria1 : bash — Konsole
Archivo  Editar  Ver  Marcadores  Preferencias  Ayuda
leo@leo-desktop: ~/proyectosDotnet/teoria1$ dotnet run
```

Compilar y ejecutar la aplicación con
el comando `dotnet run`
Este comando asume que el proyecto
está en la carpeta actual

Ejecución de la aplicación



The screenshot shows a terminal window titled "teoria1 : bash — Konsole". The window has a menu bar with "Archivo", "Editar", "Ver", "Marcadores", "Preferencias", and "Ayuda". The terminal content shows the following sequence of commands and output:

```
leo@leo-desktop:~/proyectosDotnet/teoria1$ dotnet run
Hello World!
leo@leo-desktop:~/proyectosDotnet/teoria1$
```

A white arrow points from the text "La aplicación generada por el comando `dotnet new console` es la clásica HelloWorld. El código fuente está en el archivo `Program.cs`" to the "Hello World!" output line.



Descargar e instalar Visual Studio Code (<https://code.visualstudio.com/>)

 Visual Studio Code Docs Updates Blog API Extensions FAQ [Download](#)

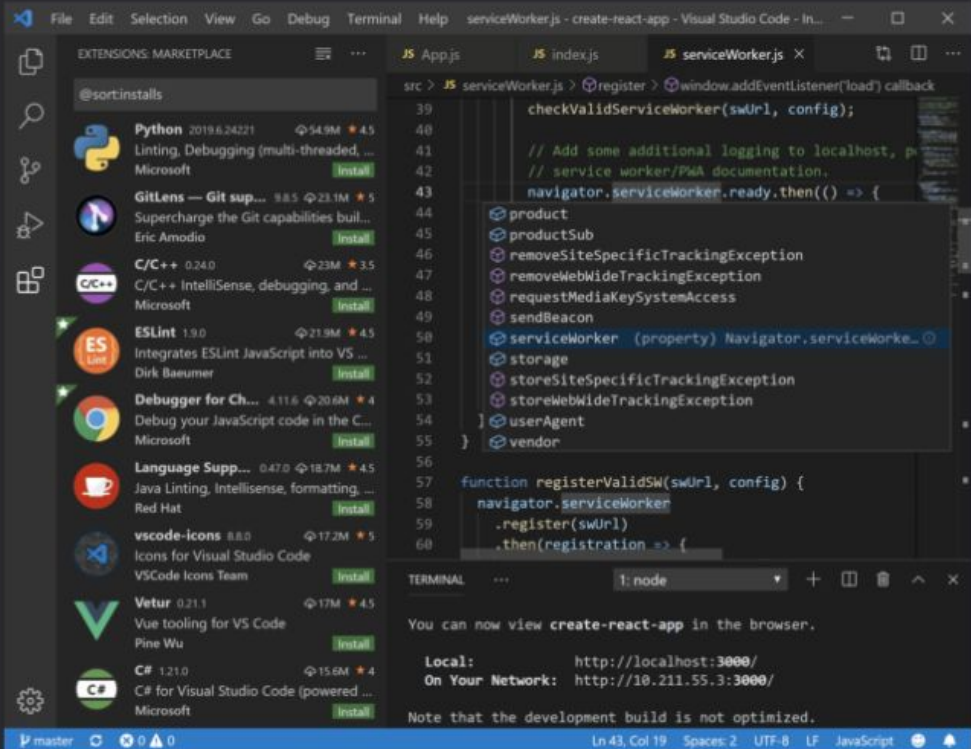
Version 1.48 is now available! Read about the new features and fixes from July.

Code editing. Redefined.

Free. Built on open source. Runs everywhere.

[Download for Windows](#)
Stable Build

		Stable	Insiders
macOS	Package	↓	↓
Windows x64	User Installer	↓	↓
Linux x64	.deb .rpm	↓ ↓	↓ ↓
Other downloads			



The screenshot shows the Visual Studio Code interface. On the left, the 'EXTENSIONS: MARKETPLACE' sidebar is open, displaying a list of installed and available extensions. The main editor area shows a JavaScript file named 'serviceWorker.js' with code for a service worker. The bottom panel shows the 'TERMINAL' with the command 'node' and the output of a web server running on localhost:3000.



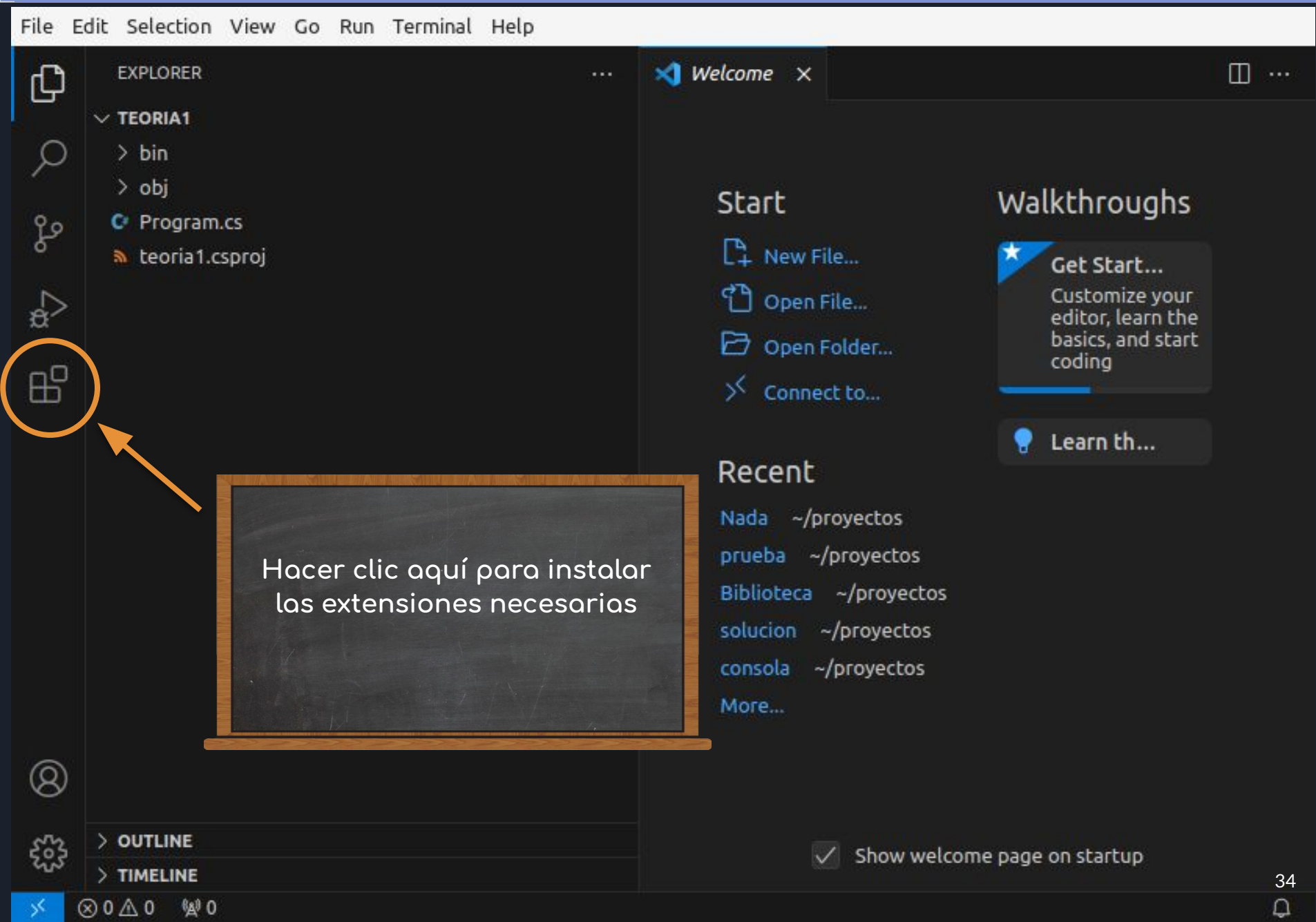
En la terminal del sistema operativo hacer:

```
teoria1 : bash — Konsole
Archivo  Editar  Ver  Marcadores  Preferencias  Ayuda
leo@leo-desktop:~/proyectosDotnet/teoria1$ dotnet run
Hello World!
leo@leo-desktop:~/proyectosDotnet/teoria1$ code .
```

El punto (.) indica la carpeta actual

Abrir Visual Studio Code en la carpeta actual

Introducción a C# - Primera aplicación en C#



The screenshot shows the Visual Studio IDE interface. The Explorer sidebar on the left is active, displaying a project named 'TEORIA1' with folders 'bin' and 'obj', and files 'Program.cs' and 'teoria1.csproj'. The Extensions icon, represented by a square with four smaller squares, is circled in orange in the sidebar. An orange arrow points from this icon to a text box that reads: 'Hacer clic aquí para instalar las extensiones necesarias'. The main editor area shows a 'Welcome' page with sections for 'Start' (New File..., Open File..., Open Folder..., Connect to...), 'Walkthroughs' (Get Start..., Learn th...), and 'Recent' (Nada, prueba, Biblioteca, solucion, consola, More...). The status bar at the bottom shows '0' errors, '0' warnings, and '0' messages.

File Edit Selection View Go Run Terminal Help

EXPLORER

TEORIA1

- > bin
- > obj
- Program.cs
- teoria1.csproj

Start

- New File...
- Open File...
- Open Folder...
- Connect to...

Walkthroughs

- Get Start...
Customize your editor, learn the basics, and start coding
- Learn th...

Recent

- Nada ~/proyectos
- prueba ~/proyectos
- Biblioteca ~/proyectos
- solucion ~/proyectos
- consola ~/proyectos
- More...

OUTLINE

TIMELINE

Show welcome page on startup

Hacer clic aquí para instalar las extensiones necesarias

Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help

EXTENSIONS

Search Extensions in Marketplace

✓ INSTALLED

> POPULAR

> RECOMMENDED

Escribir C#

para buscar las extensiones
que nos interesan

Welcome

Walkthroughs



Get Start...

Customize your
editor, learn the
basics, and start
coding



Learn th...

New File...

Open File...

Open Folder...

Connect to...

Recent

Nada ~/proyectos

prueba ~/proyectos

Biblioteca ~/proyectos

solucion ~/proyectos

consola ~/proyectos

More...



Show welcome page on startup

Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help

EXTENSIONS: MARKETPLACE

C#



C# Dev Kit

Official C# extension from Microsoft

Microsoft

1.8M ★ 2.5

Install



C#

Base language support for C#

Microsoft

25.4M ★ 3

Install



[DEPRECATED] C# Format R...

[DEPRECATED] Fix format of usings / inden...

Przemysław Orłowski

3.3M ★ 2.5



C# Extensions

C# IDE Extensions for VSCode

jchannon

1.6M ★ 4

Install



IntelliCode for C# Dev Kit

AI-assisted development For C# Dev Kit

Microsoft

1.8M ★ 2.5

Install



C# XML Documentation Co...

Generate C# XML documentation comment...

Keisuke Kato

1.1M ★ 4.5

Install



C# Extensions

796K ★ 4.5

Extension: C# Dev Kit



C# Dev Kit v1

Microsoft [microsoft.co](#)

Official C# extension f...

Install



Instalar la extensión C# Dev Kit

Se instalarán además tres extensiones más

C# Dev Kit helps you manage your code with a solution explorer and test your code with integrated unit test discovery and execution, elevating your C# development experience wherever you like to develop (Windows, macOS, Linux, and even in a Codespace).

This extension builds on top of the great C# language capabilities provided by the C#

Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help



EXTENSIONS



Search Extensions in Marketplace



INSTALLED

4



.NET Install Tool

This extension installs and manages differe...

Microsoft

137ms



C#

Base language support for C#

Microsoft

135ms



C# Dev Kit

Official C# extension from Microsoft

Microsoft

151ms



Al finalizar la instalación
tendremos estas 3
extensiones instaladas



> POPULAR

> RECOMMENDED

2

PROBLEMS

OUTPUT

...

.NET Runtime

...

...

...

...

...

ms-dotnettools.csdevkit requested to download
the .NET Runtime.

Downloading .NET version(s) 7.0.

14~x64 Done!

.NET 7.0.14~x64 executable path: /home/leo/.

config/Code/User/globalStorage/ms-dotnettools. 37



0 0 0

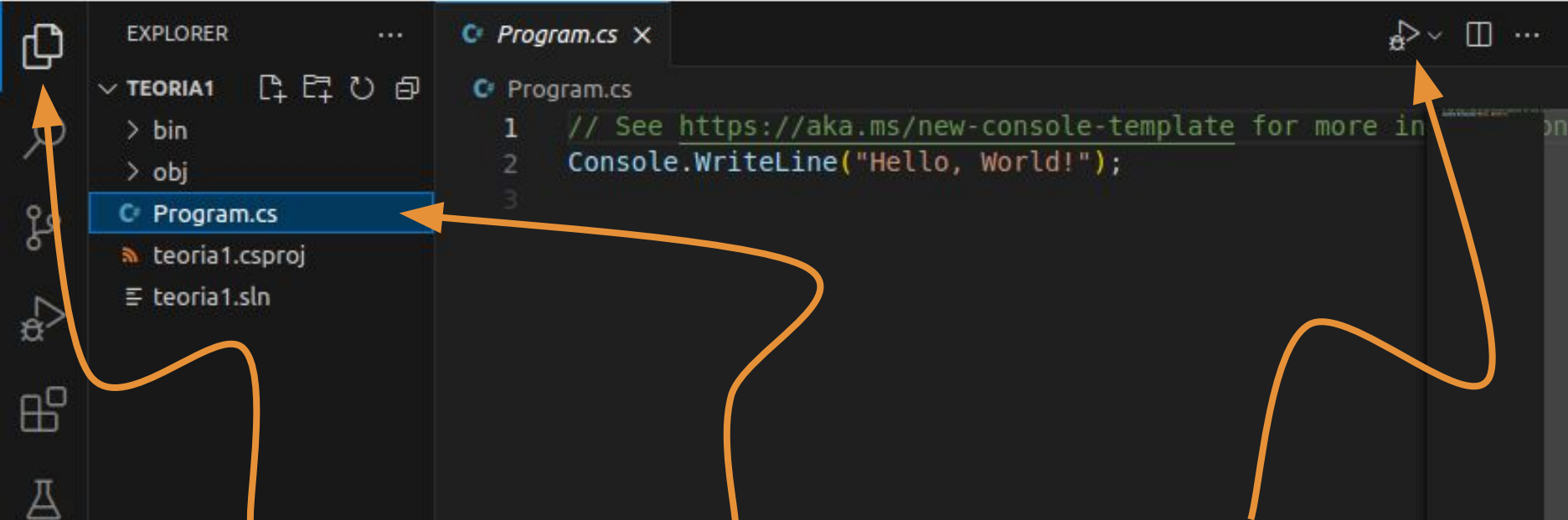


Projects: 1



Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help



(1)

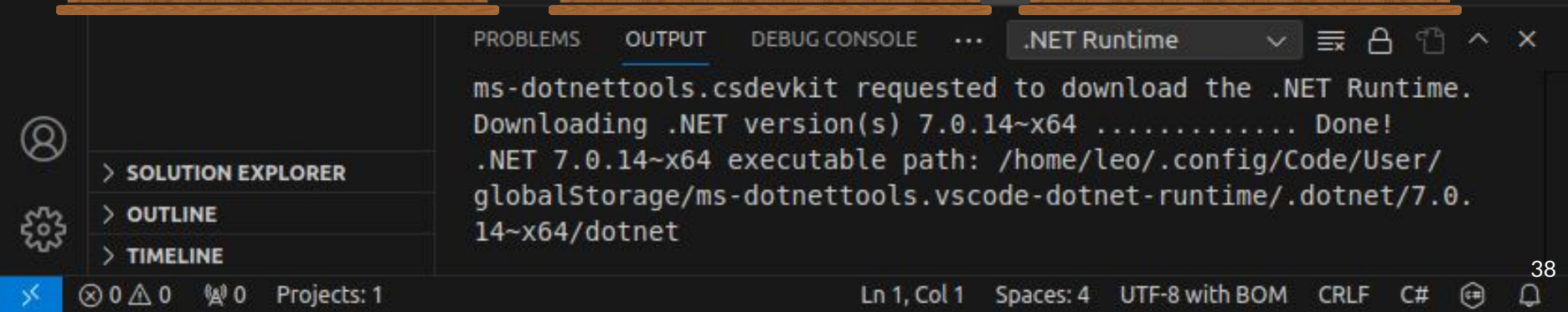
Hacer clic aquí para ver el explorador de la solución

(2)

Luego hacer clic en Program.cs

(3)

Finalmente hacer clic aquí para compilar y ejecutar



Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help

EXPLORER

- TEORIA1
 - bin
 - obj
 - Program.cs
 - teoria1.csproj
 - teoria1.sln

SOLUTION EXPLORER

- OUTLINE
- TIMELINE

Program.cs

```
1 // See https://aka.ms/new-console-te
2 Console.WriteLine("Hello, World!");
3 |
```

Mientras se compila y ejecuta la aplicación, se harán visible estos controles para hacer debug

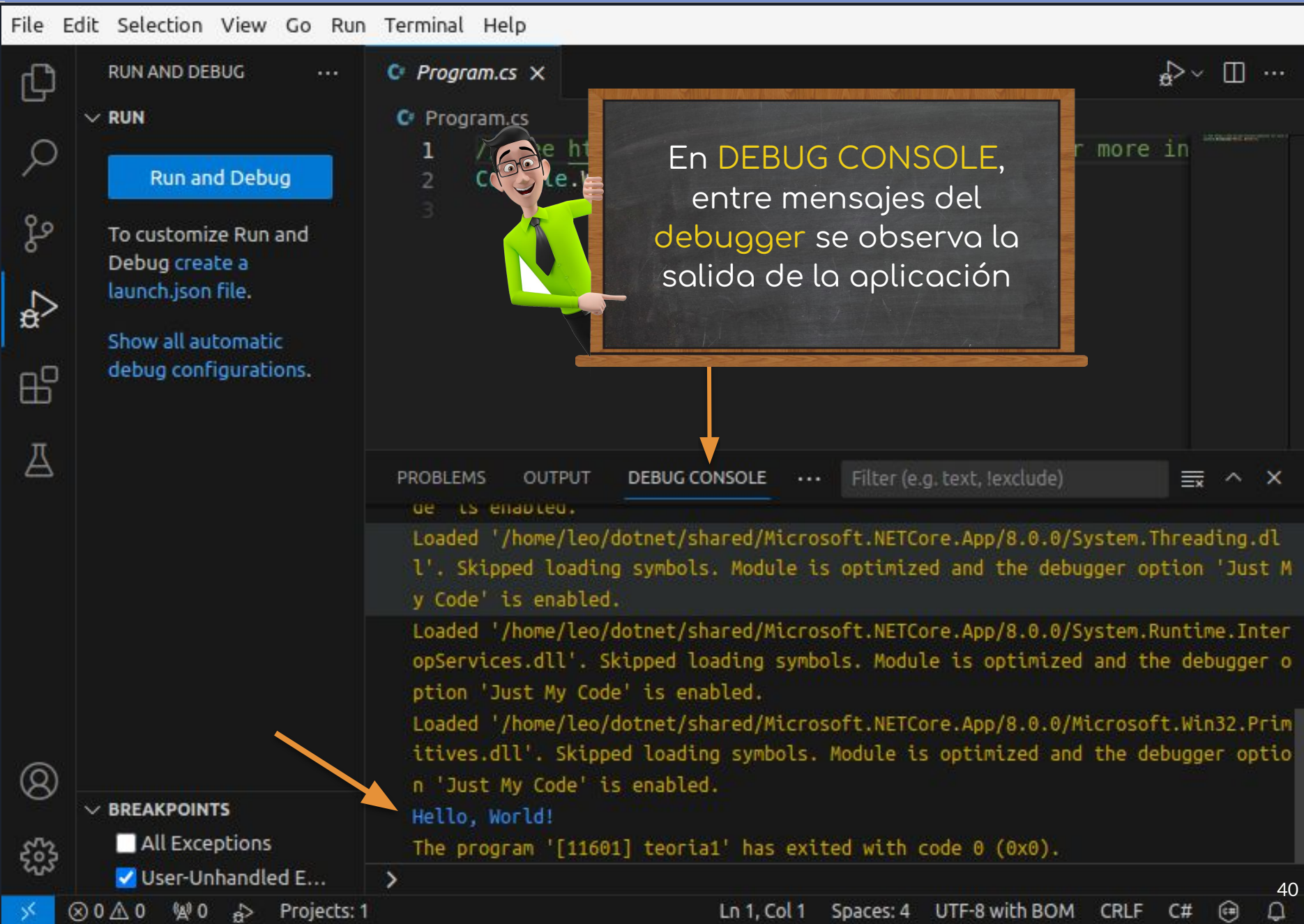
PROBLEMS DEBUG CONSOLE ... C#: teoría1 |

Filter (e.g. text, !exclude)

ule is optimized and the debugger option 'Just My Code' is enabled.

Ln 3, Col 1 Spaces: 4 UTF-8 with BOM CRLF C#

Introducción a C# - Primera aplicación en C#



File Edit Selection View Go Run Terminal Help

RUN AND DEBUG ...

Program.cs X

Program.cs

1 /...e ht
2 Co...e.k
3

Run and Debug

To customize Run and Debug create a [launch.json](#) file.

Show all automatic debug configurations.

DEBUG CONSOLE Filter (e.g. text, lexclude)

de...ls enabled.

Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/System.Threading.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/System.Runtime.InteropServices.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/Microsoft.Win32.Primitives.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

Hello, World!

The program '[11601] teoria1' has exited with code 0 (0x0).

BREAKPOINTS

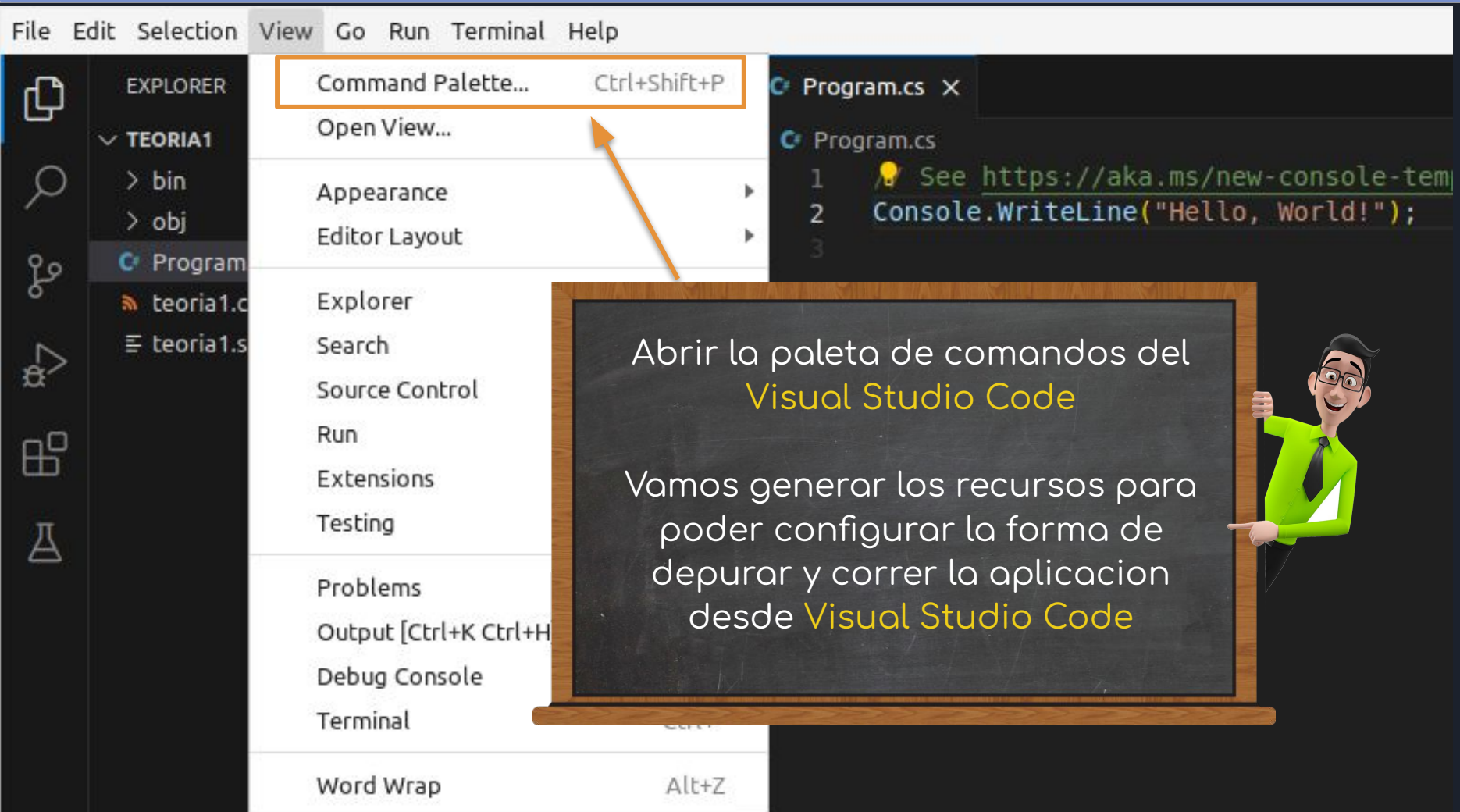
☐ All Exceptions

☒ User-Unhandled E...

Projects: 1

Ln 1, Col 1 Spaces: 4 UTF-8 with BOM CRLF C#

Introducción a C# - Primera aplicación en C#



The screenshot shows the Visual Studio Code interface. The 'View' menu is open, and the 'Command Palette...' option is highlighted with an orange box. An orange arrow points from this option to a chalkboard overlay. The chalkboard contains text in Spanish. To the right of the chalkboard, a cartoon character in a green shirt and glasses is pointing at the text. The background code editor shows a C# file named 'Program.cs' with the following code:

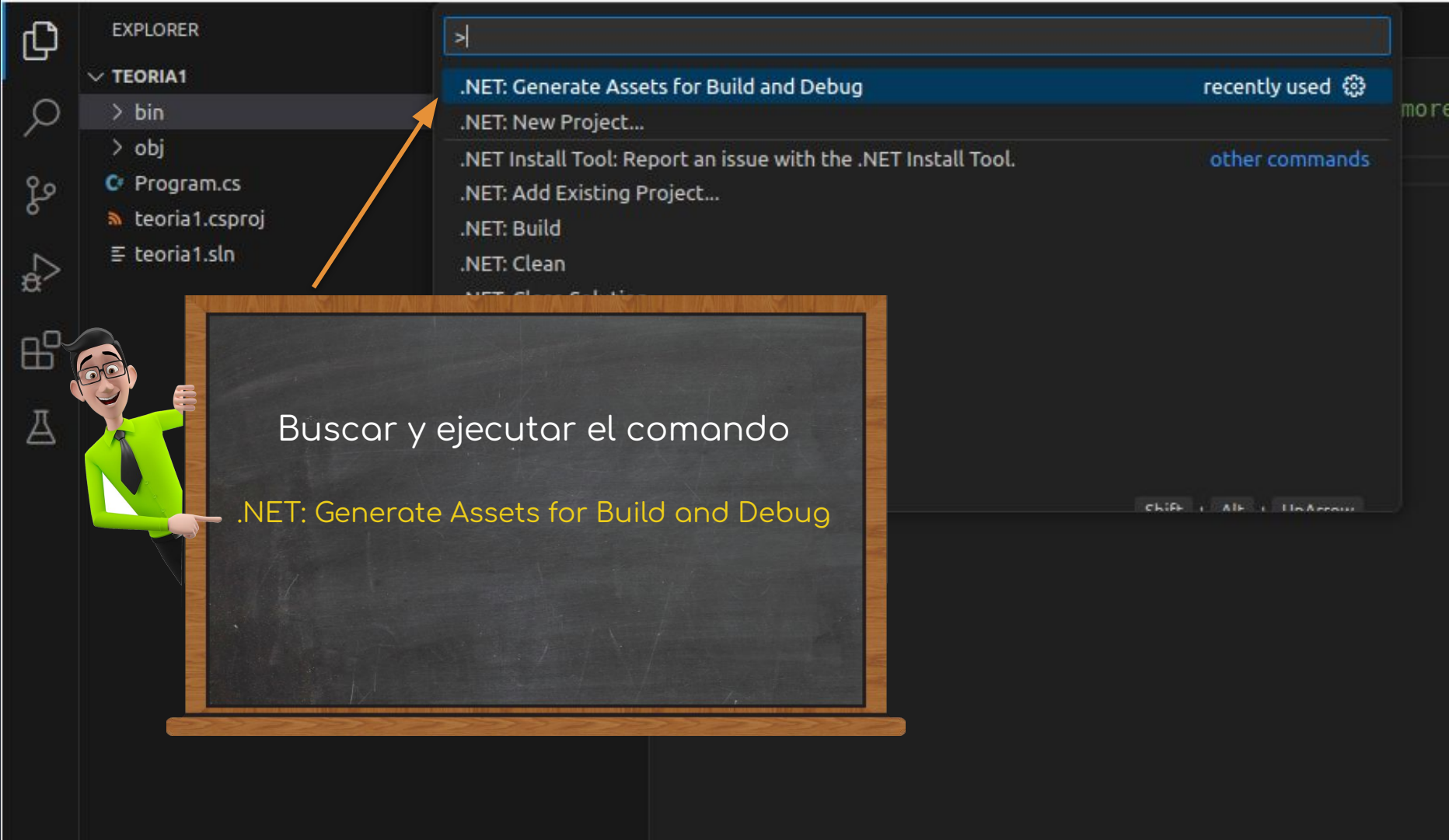
```
1  See https://aka.ms/new-console-template
2  Console.WriteLine("Hello, World!");
3
```

Abrir la paleta de comandos del Visual Studio Code

Vamos generar los recursos para poder configurar la forma de depurar y correr la aplicación desde Visual Studio Code

Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help



The image shows a screenshot of the Visual Studio IDE. On the left, the 'EXPLORER' pane shows a project named 'TEORIA1' with files 'bin', 'obj', 'Program.cs', 'teoria1.csproj', and 'teoria1.sln'. An orange arrow points from the 'bin' folder to the command palette. The command palette is open, showing a list of .NET CLI commands. The first command, '.NET: Generate Assets for Build and Debug', is highlighted. Below the list, there are links for 'recently used' and 'other commands'. In the foreground, a cartoon character in a green shirt and tie is pointing to a chalkboard. The chalkboard has the text 'Buscar y ejecutar el comando' and '.NET: Generate Assets for Build and Debug' written on it.

EXPLORER

TEORIA1

- bin
- obj
- Program.cs
- teoria1.csproj
- teoria1.sln

.NET: Generate Assets for Build and Debug recently used

.NET: New Project...

.NET Install Tool: Report an issue with the .NET Install Tool. other commands

.NET: Add Existing Project...

.NET: Build

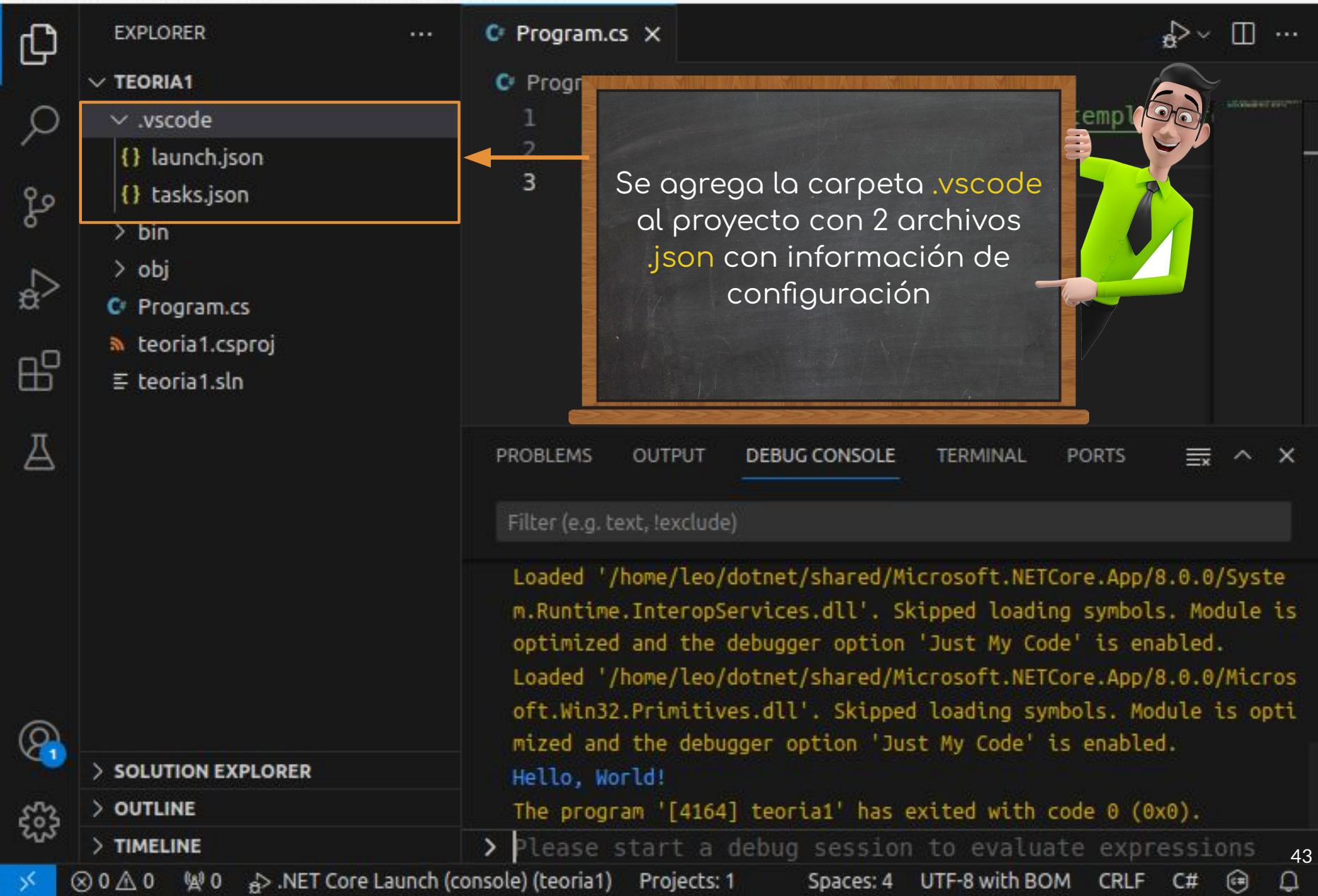
.NET: Clean

Buscar y ejecutar el comando

.NET: Generate Assets for Build and Debug

Introducción a C# - Primera aplicación en C#

File Edit Selection View Go Run Terminal Help



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar is open, showing a project named 'TEORIA1'. Under the '.vscode' folder, two files are listed: 'launch.json' and 'tasks.json'. An orange box highlights these files, and an orange arrow points from a text box to them. The text box, which also contains a cartoon character of a man in a green shirt and glasses, says: 'Se agrega la carpeta .vscode al proyecto con 2 archivos .json con información de configuración'. The main editor area shows a file named 'Program.cs' with three lines of code. The bottom panel shows the 'DEBUG CONSOLE' tab, which contains the following output: 'Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/System.Runtime.InteropServices.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled. Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/Microsoft.Win32.Primitives.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled. Hello, World! The program '[4164] teoria1' has exited with code 0 (0x0).'. The status bar at the bottom shows the current file is 'Program.cs', the project is 'teoria1', and the language is 'C#'. The status bar also shows '0 errors, 0 warnings, 0 messages' and 'Please start a debug session to evaluate expressions'.

EXPLORER

TEORIA1

- .vscode
 - launch.json
 - tasks.json
- bin
- obj
- Program.cs
- teoria1.csproj
- teoria1.sln

SOLUTION EXPLORER

OUTLINE

TIMELINE

Program.cs

```
1
2
3
```

Se agrega la carpeta .vscode al proyecto con 2 archivos .json con información de configuración

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, lexclude)

Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/System.Runtime.InteropServices.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.
Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/Microsoft.Win32.Primitives.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.
Hello, World!
The program '[4164] teoria1' has exited with code 0 (0x0).

Please start a debug session to evaluate expressions

0 0 0 .NET Core Launch (console) (teoria1) Projects: 1 Spaces: 4 UTF-8 with BOM CRLF C#

Introducción a C# - Primera aplicación en C#

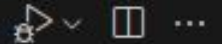
File Edit Selection View Go Run Terminal Help



EXPLORER



Program.cs x



TEORIA1

.vscode

{ } launch.json

{ } tasks.json

> bin

> obj

Program.cs

teoria1.csproj

teoria1.sln

Program.cs

```
1 // See https://aka.ms/new-console-template for
2 Console.WriteLine("Hello, World!");
```

Ahora, presionando

F5

se compila y ejecuta la aplicación

Ya no es necesario estar visualizando
Program .cs

NAL PORTS

Microsoft.NETCore.App/8.0.0/Syste
loading symbols. Module is
'Code' is enabled.

Loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/Micros
oft.Win32.Primitives.dll'. Skipped loading symbols. Module is opti
mized and the debugger option 'Just My Code' is enabled.

Hello, World!

The program '[4164] teoria1' has exited with code 0 (0x0).

> Please start a debug session to evaluate expressions

44



0 0 0



.NET Core Launch (console) (teoria1)

Projects: 1

Spaces: 4

UTF-8 with BOM

CRLF

C#



Archivo `Program.cs` en la versión `.NET 5`

```
using System;

namespace teoria1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

Hasta la versión del framework `.NET 5` la plantilla de consola generaba este código



Archivo `Program.cs` en la versión .NET 5

```
using System;
```

Espacio de nombre
`teoria1` coincidente con
el nombre del proyecto

```
namespace teoria1
```

```
{
```

```
class Program
```

```
{
```

```
static void Main(string[] args)
```

```
{
```

```
    Console.WriteLine("Hello World!");
```

```
}
```

```
}
```

```
}
```

Clase `Program` que
contiene el método
principal

Método especial denominado `Main` que
represente el punto de inicio de la
aplicación

Archivo `Program.cs` a partir de `.NET 6`

```
// See https://aka.ms/new-console-template for more information  
Console.WriteLine("Hello, World!");
```

La doble barra (//)
indica que la línea es un
comentario

Con `.NET 6`, el nuevo estilo para la clase `Program` usa características recientes de `C#` que nos permiten escribir sólo el cuerpo del método `Main`

Esta característica se conoce como
"Instrucciones de nivel superior"



Aplicación HelloWorld generada

```
Console.WriteLine("Hello World!");
```

Es una
clase

Es un método.

Argumento pasado
al método **Writeline**

Se está invocando el método **WriteLine** de la clase **Console** pasando como parámetro el **string** que se desea imprimir en la consola.





Editar `teoria1.csproj`. Cambiar `enable` por `disable` en `ImplicitUsings`

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>disable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```





Presionar **F5** para compilar y ejecutar

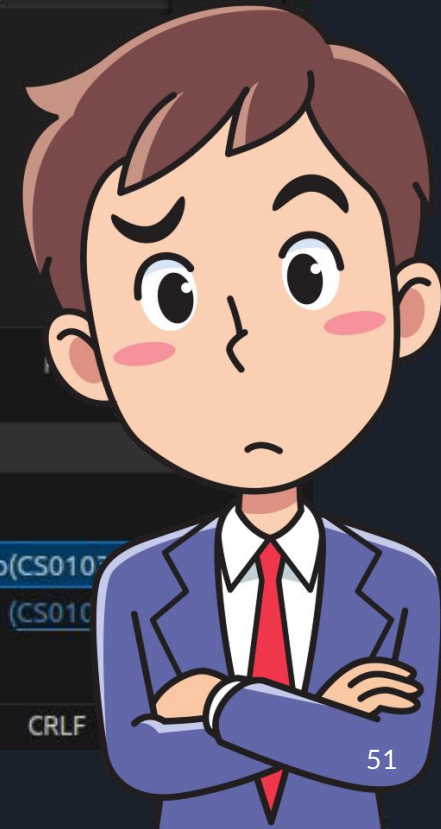
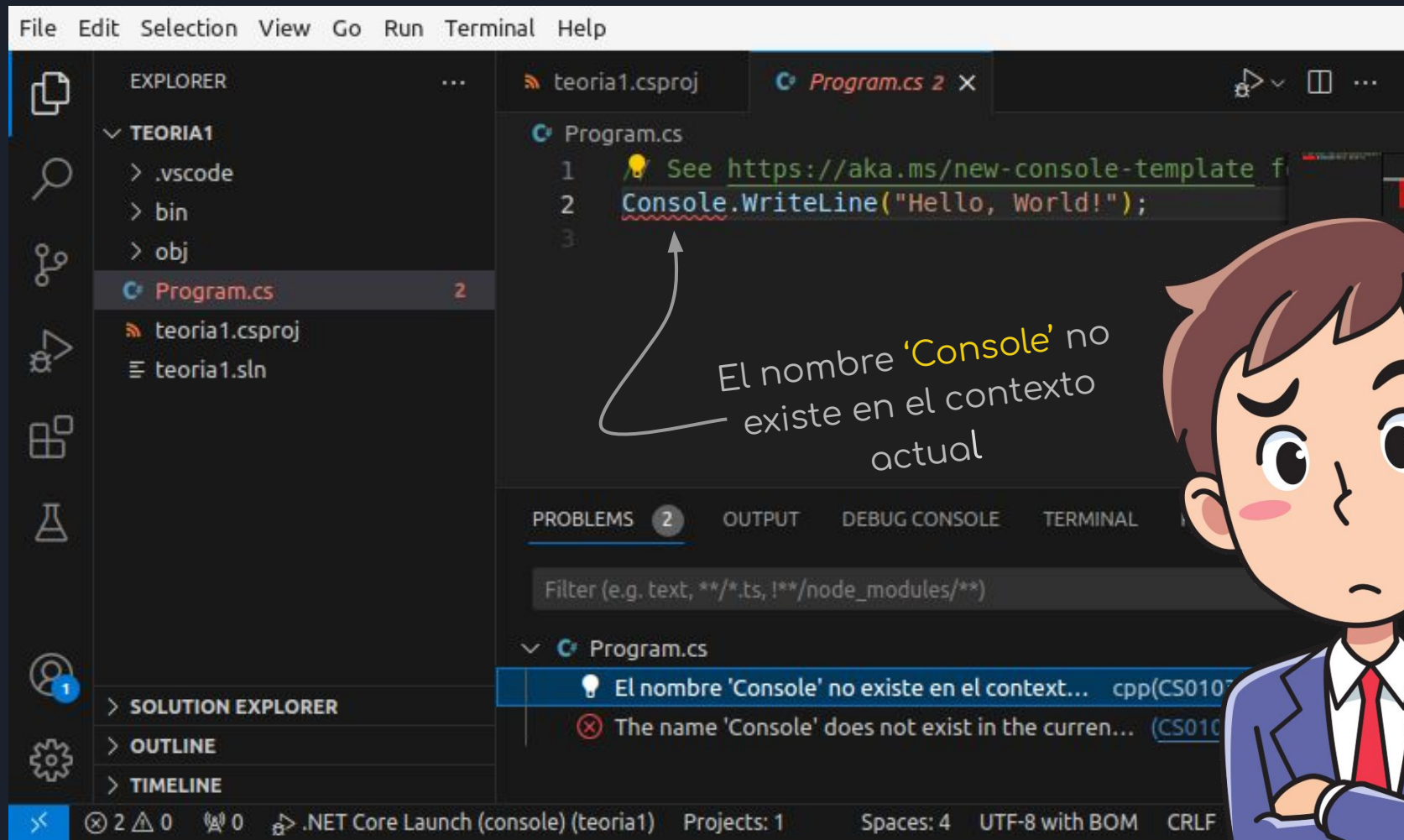
```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>disable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```



Error de compilación!





Editar Program.cs, agregar la directiva **using System** y volver a probar

```
using System;
```



```
// See https://aka.ms/new-console-template for more ...
```

```
Console.WriteLine("Hello, World!");
```



```
using System;
```

```
// See https://aka.ms/new-console-template for more information
```

```
Console.WriteLine("Hello, World!");
```





Modificar nuevamente **Program.cs** de la siguiente manera y probar nuevamente

```
// See https://aka.ms/new-console-template for more ...  
System.Console.WriteLine("Hello, World!");
```

Eliminar la directiva **using**
System pero anteponer **System.**
a la clase **Console**



```
// See https://aka.ms/new-console-template for more information  
System.Console.WriteLine("Hello, World!");
```





Habilitar nuevamente **ImplicitUsings** en **teoria1.csproj**.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```



Directiva `using`

- El `nombre extenso` de una clase (o de cualquier otro tipo) lleva como prefijo el espacio de nombres en la que se ha definido.
- El `nombre extenso` de la clase `Console` es `System.Console` porque está definida en el espacio de nombres `System`.
- Si usamos la directiva `using System` al inicio del código podemos referirnos a cualquier tipo definido en el espacio de nombres `System` sin anteponer el prefijo `System`.

Directiva `using`

- El alcance de la directiva `using` es el archivo fuente donde se especifica.
- Con `C#10` (liberado con `.NET 6`) es posible utilizar la directiva `global using` para extender el alcance a todos los archivos fuente del proyecto.
- Si en el archivo de configuración del proyecto (extensión `.csproj`) colocamos:

```
<ImplicitUsings>enable</ImplicitUsings>
```

estamos habilitando las directivas de `using` implícitas

Directivas `using` implícitas (incorporadas en .NET 6)

El compilador agrega automáticamente un conjunto de directivas `using` en función del tipo de proyecto. En el caso de las aplicaciones de consola se incluye;

- `using System;`
- `using System.IO;`
- `using System.Collections.Generic;`
- `using System.Linq;`
- `using System.Net.Http;`
- `using System.Threading;`
- `using System.Threading.Tasks;`

Otros tipos de aplicación incluyen más espacios de nombres que son comunes para esos tipos de aplicación.

Tipos integrados y operadores



Tipos integrados en C#

- C# proporciona un conjunto estándar de **tipos numéricos** integrados para representar números enteros y valores de punto flotante
- Además de los tipos numéricos, C# proporciona tipos integrados para representar **expresiones booleanas**, **caracteres de texto**, **cadenas de texto** y **objetos**. El tipo **dynamic**, que veremos luego, también se considera integrado a C#
- Todos los tipos integrados de C# son tipos de **.NET** y pueden referenciarse por el nombre del tipo en la plataforma o por un **alias** propio de C# (a excepción de **dynamic**)



Tipos enteros integrados en C#

Alias de C#	Intervalo	Tamaño	Tipo de .NET
sbyte	De -128 a 127	8 bits con signo	System.SByte
byte	De 0 a 255	8 bits sin signo	System.Byte
short	De -32.768 a 32.767	16 bits con signo	System.Int16
ushort	De 0 a 65.535	16 bits sin signo	System.UInt16
int	De -2.147.483.648 a 2.147.483.647	32 bits con signo	System.Int32
uint	De 0 a 4.294.967.295	32 bits sin signo	System.UInt32
long	De -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	64 bits con signo	System.Int64
ulong	De 0 a 18.446.744.073.709.551.615	64 bits sin signo	System.UInt64



Tipos de punto flotante integrados en C#

Alias C#	Intervalo aproximado	Tamaño	Tipo de .NET	Precisión
float	De $\pm 1,5 \times 10^{-45}$ a $\pm 3,4 \times 10^{38}$	32 bits	System.Single	6 a 9 dígitos aprox.
double	De $\pm 5,0 \times 10^{-324}$ a $\pm 1,7 \times 10^{308}$	64 bits	System.Double	15 a 17 dígitos aprox.
decimal	De $\pm 1,0 \times 10^{-28}$ a $\pm 7,9228 \times 10^{28}$	128 bits	System.Decimal	6 a 9 dígitos aprox.



Demás tipos integrados en C#

Alias C#	Tipo de .NET
char	System.Char
bool	System.Boolean
string	System.String
object	System.Object
dynamic	System.Object

C# es sensible a mayúsculas y minúsculas

¡Atención!

C# es sensible a las mayúsculas y minúsculas, por lo tanto `bool` es un tipo válido de C#, en cambio `Bool` es inválido.

Observar que los alias de C# para los tipos integrados comienzan siempre en minúscula





Tipos integrados en C#

- Los **tipos integrados** son estructuras, salvo **string** y **object** que son clases
- Están definidos en el espacio de nombres **System**
- Por lo tanto, si se utiliza la directiva **using System** es posible referirse al tipo **System.Boolean** simplemente como **Boolean**
- Sin embargo, para el caso de los tipos integrados, lo usual es utilizar el **alias** definido para el lenguaje correspondiente

Declaración de variables y constantes

```
// Declaración variable de tipo char  
char a;
```

```
//Declaración y asignación de variable de tipo double  
double e = 5.2;
```

```
//declaración y asignación de constante  
const double pi = 3.1416;
```

```
// Declaración múltiple variables de tipo int  
int b, c, d;
```

```
//declaración y asignación múltiple variables de tipo int  
int f = 2, g = 3;
```

Las declaraciones múltiples no suelen utilizarse

Declaración de variables y constantes

- **Identificadores reservados:** Como en todo lenguaje, existen en C# palabras claves que son identificadores reservados que tienen un significado especial para el compilador y que por lo tanto no podemos utilizar como nombres de variables (**if, for, this, class, static, new, switch, interface, int, double, string, event**, etc.)
- Es posible utilizar estas palabras claves como identificadores de variables anteponiendo el carácter @. Ejemplos:

```
int @this = 1;  
string @for = "hola mundo";  
double @double = 1.2;
```

Literales

```
// Comillas simples para un literal de tipo char  
char a = 'A';
```

```
// Comillas dobles para un literal de tipo string  
string st = "hola mundo!";
```

```
// Un literal numérico sin punto decimal es int por defecto  
int i = 27;
```

```
// El prefijo 0x se utiliza con una expresión hexadecimal  
int j = 0xFF; // a j se le asigna el valor 255
```

```
// El prefijo 0b se utiliza con una expresión binaria  
int n = 0b10110; // a n se le asigna el valor 22
```

```
// Un literal numérico con punto decimal es double por defecto  
double pi = 3.1416;
```



Codificar las siguientes líneas

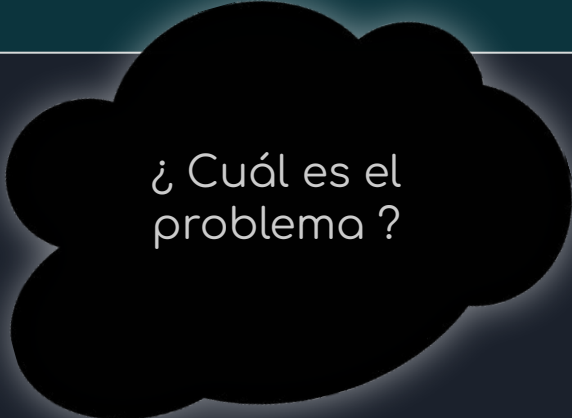
```
double d = 15.1;  
float f = 21.2;
```





Codificar las siguientes líneas

```
double d = 15.1;  
float f = 21.2;
```



¿Cuál es el problema ?

Mmm..., hay algo mal que no anda bien

Literales

```
double d = 15.1;
```

```
float f = 21.2;
```

Un literal con punto decimal es por defecto de tipo **double**, por lo tanto no puede asignarse a una variable de tipo **float**



Literales

```
double d = 15.1;
```

```
float f = 21.2f;
```

Se soluciona agregando el
sufijo **f** al literal.
21.2f es un literal de tipo
float



Observar que ...

```
int i = 15;  
byte b = 21;
```

No se necesita
ningún sufijo

No ocurre lo mismo
para el caso de
literales enteros.

Sufijos para literales

- Literales enteros
 - Sin sufijo: `int` (ejemplo 73)
 - **L**, **l**: `long`
 - **U**, **u**: `unsigned`
- Literales reales
 - Sin sufijo: `double` (ejemplo 73.5)
 - **F**, **f**: `float`
 - **D**, **d**: `double`
 - **M**, **m**: `decimal`

Operadores aritméticos

Operador	Operación
+	Suma
-	Resta
*	Multiplicación
/	División
%	Residuo

```
double resultado = 1/2;
```



¡ Cuidado con la división !

O los resultados pueden
ser dolorosos

Cuidado con la división

Probar, analizar y responder sobre el siguiente código para la próxima clase: ¿Por qué las variables **r1** y **r2** difieren?

```
double r1 = 17 / 3;  
double r2 = 17 / 3.0;  
Console.WriteLine(r1);  
Console.WriteLine(r2);
```



Operadores relacionales

Operador	Operación
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
!=	Diferente de
==	Igual a

Operadores lógicos

Operador	Operación
&	AND
	OR
!	NOT
^	XOR
&&	AND en cortocircuito
	OR en cortocircuito

Operadores `&`, `&&`, `|` y `||`

Probar, analizar y responder para la próxima clase sobre el código siguiente:

1. ¿Cuál es la salida por consola?
2. ¿si se reemplaza `int x = 2;` por `int x = 0;`?

```
int x = 2;  
int y = 5;  
Console.WriteLine(x != 0 && y / x == 2);  
Console.WriteLine(x != 0 & y / x == 2);  
Console.WriteLine(x == 0 || y / x == 2);  
Console.WriteLine(x == 0 | y / x == 2);
```



Operadores de asignación

Operador	Operación
++	Incremento
--	Decremento
=	Asignación simple
*=	Multiplicación más asignación
/=	División más asignación
%=	Residuo más asignación
+=	Suma más asignación
-=	Resta más asignación

Operador incremento

Probar, analizar y responder para la próxima clase sobre el código siguiente:

1. ¿Cuál es la salida por consola?
2. ¿En qué difieren el pre y post incremento?

```
int x = 1;  
Console.WriteLine(x++); //post incremento  
Console.WriteLine(x);  
  
Console.WriteLine(++x); //pre incremento  
Console.WriteLine(x);
```



Operador decremento

Probar, analizar y responder para la próxima clase sobre el código siguiente:

1. ¿Cuál es la salida por consola?
2. ¿En qué difieren el pre y post decremento?

```
int x = 10;  
int y = x--; //post decremento  
Console.WriteLine(y);  
Console.WriteLine(x);  
  
y = --x; //pre decremento  
Console.WriteLine(y);  
Console.WriteLine(x);
```



Operadores incremento y decremento

Responder para la próxima clase
¿Cuál es la salida por consola del siguiente código?

```
int x = 10;  
Console.WriteLine(x++ == 10);  
Console.WriteLine(x-- == 10);  
Console.WriteLine(++x == 10);  
Console.WriteLine(--x == 10);
```





Ejercicio práctico: Codificar un programa que solicite al usuario ingresar por teclado su nombre y saludarlo de manera personalizada



La siguiente sentencia

```
string st = Console.ReadLine();
```

Lee un **string** desde la consola y lo asigna a la variable **st**



Ejercicio práctico: Codificar un programa que solicite al usuario ingresar por teclado su nombre y saludarlo de manera personalizada

```
Console.WriteLine("Ingrese su nombre");  
string nombre = Console.ReadLine();  
Console.WriteLine("Hola " + nombre);
```



Ejecutar en
Visual Studio Code
presionando **F5**

Ejercicio práctico: Codificar un programa que solicite al usuario ingresar por teclado su nombre y saludarlo de manera personalizada

File Edit Selection View Go Run Terminal Help

EXPLORER

PROYECTOSDOTNET

.vscode

launch.json

tasks.json

teoria1 > Program.cs

```
1 Console.WriteLine("Ingrese su nombre");  
2 string nombre = Console.ReadLine();  
3 Console.WriteLine("Hola " + nombre);
```

`Console.ReadLine()` espera que el usuario ingrese una línea de texto y presione **[Enter]**

En la **consola interna** de Visual Studio Code, la entrada de datos del usuario se realiza desde aquí

PROBLEMS 1 OUTPUT DEBUG CONSOLENET Core La

Enter (e.g. text, lexclude)

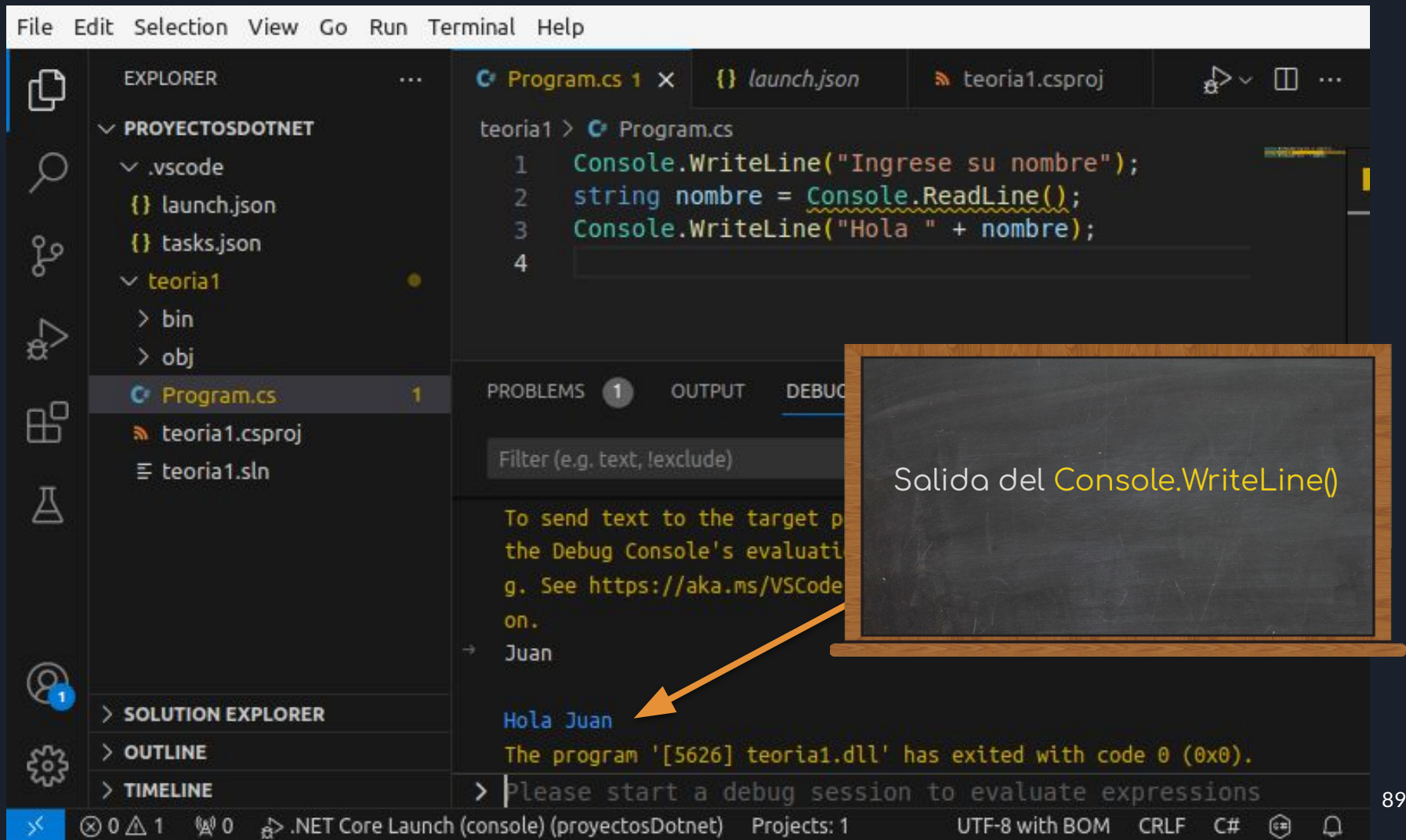
loaded '/home/leo/dotnet/shared/Microsoft.NETCore.App/8.0.0/Microsoft.Windows.Common-UI-Fonts.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

Ingrese su nombre

To send text to the target process's standard input, enter text into the Debug Console's evaluation box while the target process is running. See <https://aka.ms/VSCode-CS-LaunchJson-Console> for more information.

> Juan

Ejercicio práctico: Codificar un programa que solicite al usuario ingresar por teclado su nombre y saludarlo de manera personalizada



The screenshot shows the Visual Studio Code interface with a C# program named `Program.cs` in the `teoria1` project. The program prompts the user to enter their name and then greets them. The output window shows the program's execution, including the user input "Juan" and the output "Hola Juan". A yellow arrow points from the `Console.WriteLine` statement in the code to the output "Hola Juan". A chalkboard graphic with the text "Salida del `Console.WriteLine()`" is also present, with an arrow pointing to the same output.

```
File Edit Selection View Go Run Terminal Help
```

EXPLORER

- PROYECTOSDOTNET
 - .vscode
 - launch.json
 - tasks.json
 - teoria1
 - bin
 - obj
 - Program.cs**
 - teoria1.csproj
 - teoria1.sln

teoria1 > Program.cs

```
1 Console.WriteLine("Ingrese su nombre");
2 string nombre = Console.ReadLine();
3 Console.WriteLine("Hola " + nombre);
4
```

PROBLEMS 1 OUTPUT DEBUG

Filter (e.g. text, lexclude)

To send text to the target p... the Debug Console's evaluati... g. See <https://aka.ms/VSCode> on.

→ Juan

Hola Juan

The program '[5626] teoria1.dll' has exited with code 0 (0x0).

> Please start a debug session to evaluate expressions

Solida del `Console.WriteLine()`

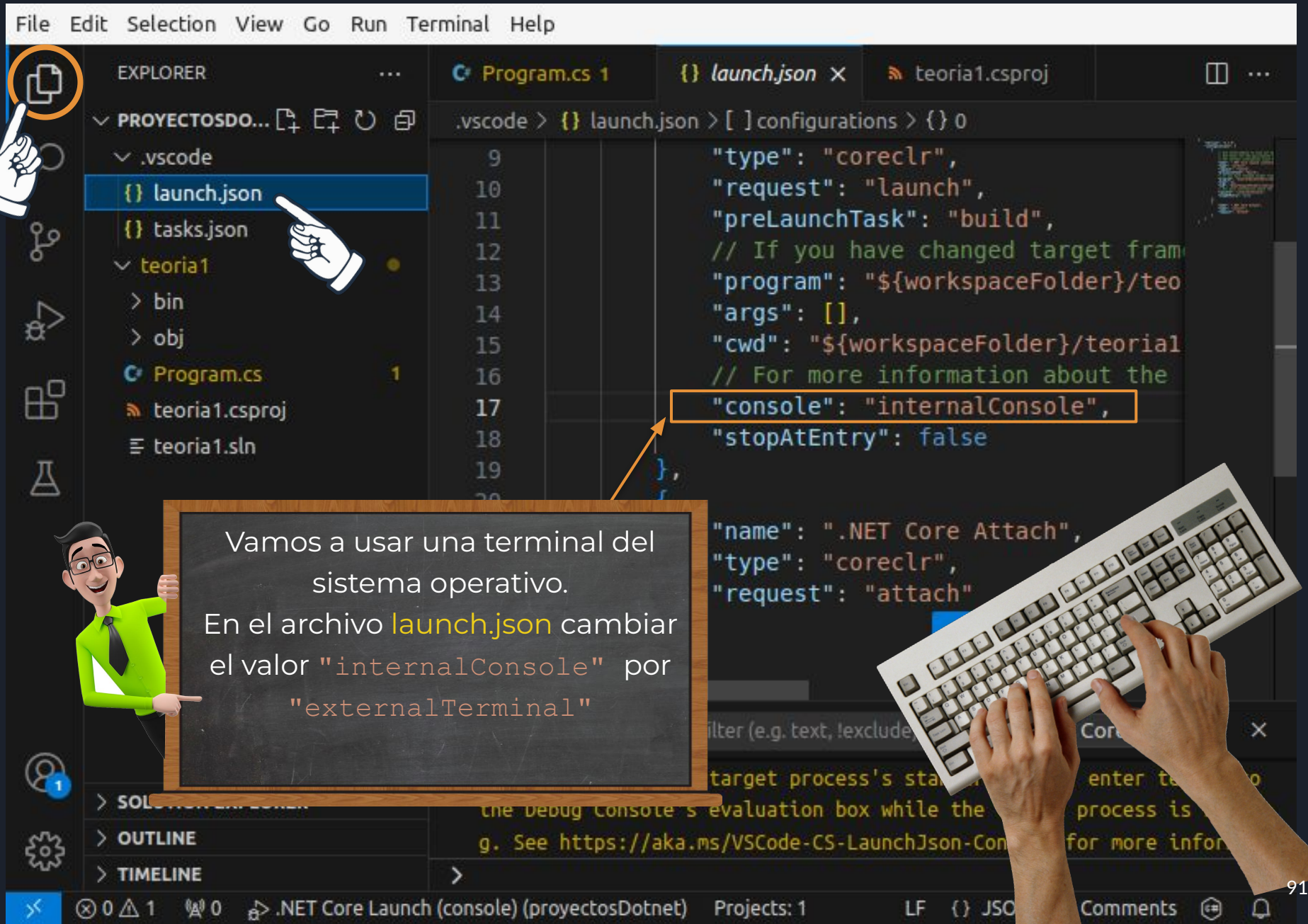
.NET Core Launch (console) (proyectosDotnet) Projects: 1 UTF-8 with BOM CRLF C#

Configuración de la terminal destino donde Visual Studio Code ejecutará la aplicación

Podemos configurar el proyecto para que **Visual Studio Code** utilice una **terminal integrada** del propio editor o una terminal del **sistema operativo**



Introducción a C# - Ejercitación



File Edit Selection View Go Run Terminal Help

EXPLORER

- PROYECTOSDO...
- .vscode
 - launch.json**
 - tasks.json
- teoria1
 - bin
 - obj
 - Program.cs
 - teoria1.csproj
 - teoria1.sln

Program.cs 1

launch.json

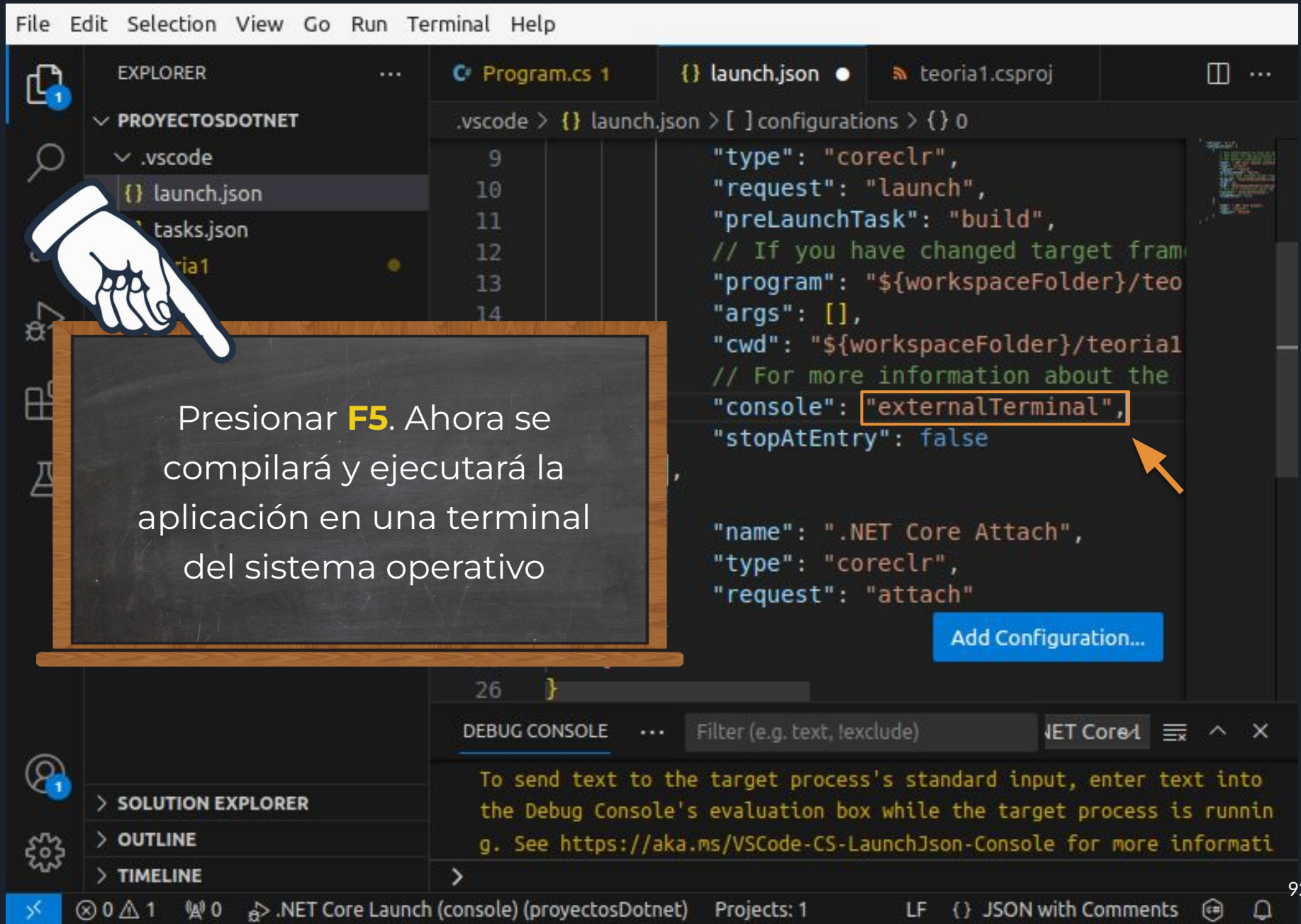
```
.vscode > {} launch.json > [ ] configurations > {} 0
9
10
11
12
13
14
15
16
17
18
19
20
    "type": "coreclr",
    "request": "launch",
    "preLaunchTask": "build",
    // If you have changed target framework
    "program": "${workspaceFolder}/teoria1/bin/Debug/netcoreapp3.1/teoria1.dll",
    "args": [],
    "cwd": "${workspaceFolder}/teoria1",
    // For more information about the
    "console": "internalConsole",
    "stopAtEntry": false

    "name": ".NET Core Attach",
    "type": "coreclr",
    "request": "attach"
```

Vamos a usar una terminal del sistema operativo.
En el archivo **launch.json** cambiar el valor `"internalConsole"` por `"externalTerminal"`

91

Introducción a C# - Ejercitación



The image shows a screenshot of the Visual Studio Code (VS Code) interface. The Explorer panel on the left shows the file structure of a project named 'PROYECTOSDOTNET', with 'launch.json' selected. The main editor area displays the 'launch.json' file, which contains JSON configuration for debugging. The 'console' property is highlighted with an orange box and labeled 'externalTerminal'. A hand icon points to a chalkboard overlay on the left side of the editor, which contains text in Spanish. The bottom panel shows the 'DEBUG CONSOLE' with a message about sending text to the target process's standard input. The status bar at the bottom indicates the current configuration is '.NET Core Launch (console) (proyectosDotnet)'.

File Edit Selection View Go Run Terminal Help

EXPLORER

PROYECTOSDOTNET

.vscode

launch.json

tasks.json

Program.cs 1

launch.json

teoria1.csproj

.vscode > {} launch.json > [] configurations > {} 0

```
{
  "type": "coreclr",
  "request": "launch",
  "preLaunchTask": "build",
  // If you have changed target framework,
  "program": "${workspaceFolder}/teoria1/bin/Debug/netcoreapp3.1/teoria1.dll",
  "args": [],
  "cwd": "${workspaceFolder}/teoria1",
  // For more information about the
  "console": "externalTerminal",
  "stopAtEntry": false

  "name": ".NET Core Attach",
  "type": "coreclr",
  "request": "attach"
}
```

Add Configuration...

26 }

DEBUG CONSOLE

Filter (e.g. text, !exclude)

.NET Core Launch (console) (proyectosDotnet)

Projects: 1

LF {} JSON with Comments

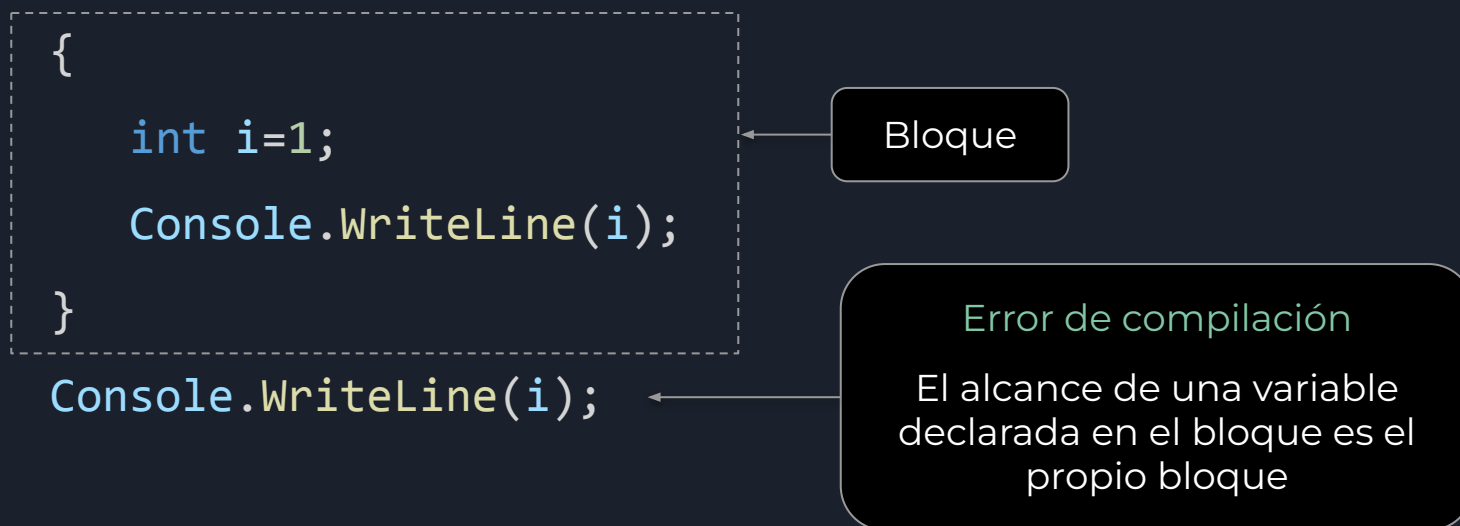
To send text to the target process's standard input, enter text into the Debug Console's evaluation box while the target process is running. See <https://aka.ms/VSCode-CS-LaunchJson-Console> for more information.

Presionar **F5**. Ahora se compilará y ejecutará la aplicación en una terminal del sistema operativo

Estructuras de control

Estructura de control: Bloque

Es una lista de cero, o más sentencias encerradas entre llaves **{ }**



Estructura de control: Bloque

No se puede ocultar el nombre de una variable en un bloque más interno (anidado)

```
int i;  
{  
    double j = 1.1;  
}  
{  
    char j = 'A';  
    int i = 1;  
}
```

Válido

Es posible volver a utilizar el nombre de una variable en un bloque hermano. Estas dos variables *j* son variables distintas

Error de compilación

La variable **i** no se puede declarar en este ámbito porque ese nombre se está usando en un ámbito local envolvente

Estructura de control: Condicional (if)

```
if (condición)
{
    //bloque que se ejecuta
    //si condición es verdadera
}
else
{
    //bloque que se ejecuta
    //si condición es verdadera
}
```

La condición es una **expresión booleana**. Va siempre entre paréntesis

la parte del **else** puede ser omitida

Operador condicional ? :

No es una estructura de control pero muchas veces nos permite evitar la utilización de un `if`

También conocido como **operador condicional ternario**.

condición `?` consiguiente `:` alternativa

Ejemplo de uso:

```
int n = 10;  
string st = (n < 0) ? "negativo" : "no negativo";  
Console.WriteLine(st);
```



Estructura de control: Switch

La expresión cuyo valor se va a utilizar buscando coincidencias en las secciones case va siempre entre paréntesis

```
switch (expresion)
{
    case valor1:
        // lista de instrucciones para
        // cuando expresion == valor1
        break;
    case valor2:
        // lista de instrucciones para
        // cuando expresion == valor2
        break;
    default:
        // lista de instrucciones para
        // cuando no hubo coincidencia
        // con ninguno de los casos anteriores
        break;
}
```

Expresión **switch**

No es una estructura de control. Se utiliza para obtener un resultado a partir de una lista de expresiones candidatas basadas en una coincidencia de patrón con una expresión de entrada

Ejemplo de uso:

```
int n = 0;  
string st = n switch { < 0 => "negativo", > 0 => "positivo", _ => "cero" };  
Console.WriteLine(st);
```



Equivalencia al
default de la
estructura de
control **Switch**

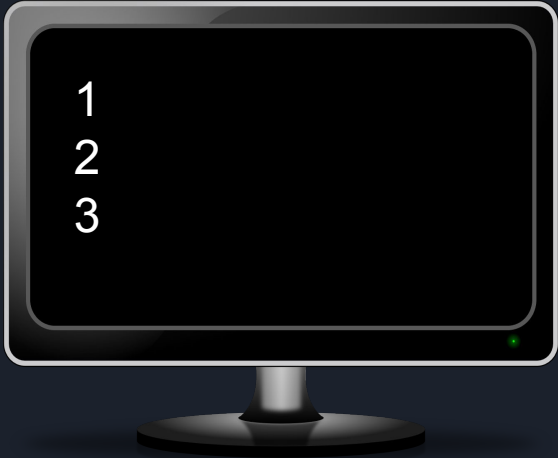
Estructura de control: **while**

```
int i = 1;  
while (i <= 3)
```

La condición siempre va
entre paréntesis

Bucle

```
{  
    Console.WriteLine(i);  
    i++;  
}
```



1
2
3

Estructura de control: **do-while**

```
int i = 1;  
do  
{  
    Console.WriteLine(i);  
    i++;  
}  
while (i <= 3);
```

Bucle

La condición siempre va
entre paréntesis

1
2
3

Estructura de control: **for**

```
for (<inicialización>; <condición>; <iterador>)  
{  
    <código del bucle>  
}
```

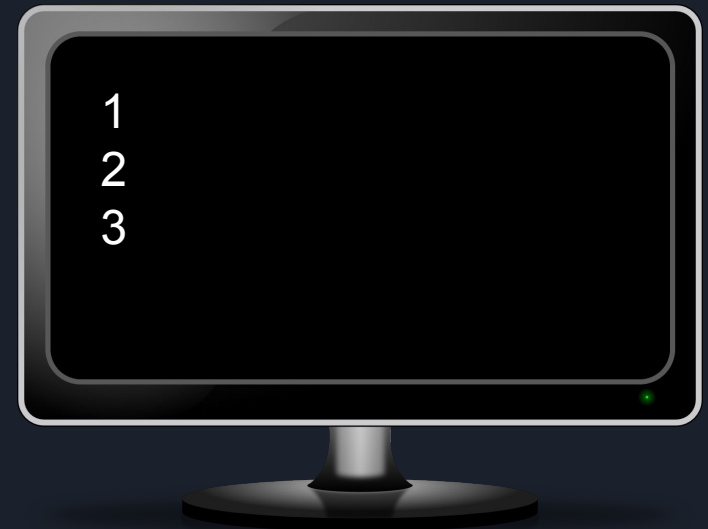
Es equivalente a:

```
<inicialización>  
while (<condición>)  
{  
    <código del bucle>  
    <iterador>  
}
```

Estructura de control: **for**

Ejemplo:

```
int i;  
for (i = 1; i <= 3; i++)  
{  
    Console.WriteLine(i);  
}
```



Estructura de control: **for**

Las secciones
del **for** pueden
omitirse

Distintas maneras de hacer lo mismo

```
int i = 1;
for (; i <= 3; i++)
{
    Console.WriteLine(i);
}
```

```
int i = 1;
for (; i <= 3; )
{
    Console.WriteLine(i++);
}
```

```
int i = 1;
for (; ; )
{
    Console.WriteLine(i++);
    if (i > 3) break;
}
```

finaliza el for

Estructura de control: **for**

Alternativa

```
for (int i = 1; i <= 3; i++)  
{  
    Console.WriteLine(i);  
}  
. . .
```

Es muy común declarar la variable en la sección de inicialización.

De esta manera, el ámbito de la variable **i** es el bloque del **for**

En este punto la variable **i** no está accesible

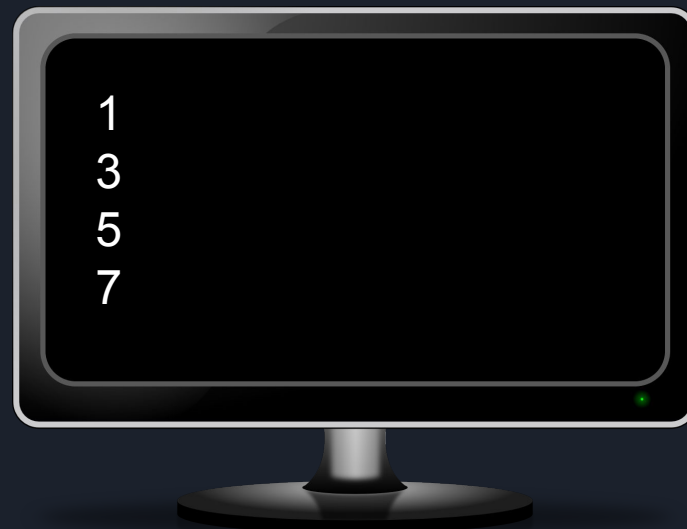
Interrupción de bucles

- **break** – El bucle termina inmediatamente
- **continue** – Termina el ciclo corriente inmediatamente (la ejecución continúa con el próximo ciclo)
- **goto** – Permite saltar a una porción del código identificada por una etiqueta **(no recomendada)**
- **return** – Finaliza el bucle y del método que lo contiene.

Interrupción de bucles

Ejemplo:

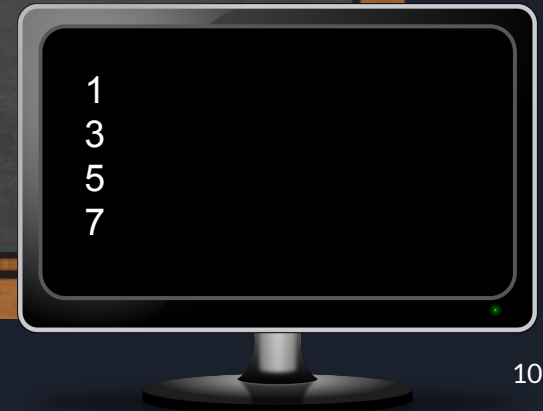
```
for (int i = 1; i <= 20; i++)  
{  
    if (i == 8)  
        break;  
    if (i % 2 == 0)  
        continue;  
    Console.WriteLine(i);  
}
```



Una mejor y más legible solución que la presentada en la diapositiva anterior



```
for (int i = 1; i <= 7; i += 2)
{
    Console.WriteLine(i);
}
```





Ejercicio práctico: Solicitar al usuario que ingrese por teclado un número **n** y calcular la **sumatoria desde 1 hasta n** utilizando la estructura de control **for**

```
string st = Console.ReadLine();  
int n = int.Parse(st);
```

La instrucción `int n = int.Parse(st)` convierte el string `st` al entero correspondiente y lo asigna a la variable `n`



Ejercicio práctico: Solicitar al usuario que ingrese por teclado un número n y calcular la sumatoria desde 1 hasta n



```
Console.WriteLine("Ingrese n");
int n = int.Parse(Console.ReadLine());
int suma = 0;
for(int i = 1; i <= n; i++)
{
    suma += i; //ídem a: suma = suma + i;
}
Console.WriteLine("Suma de 1 a " + n + " = " + suma);
```

Fin de la teoría 1

Práctica sobre la teoría 1

1) Consultar en la documentación de Microsoft y responder cuál es la diferencia entre los métodos `WriteLine()` y `Write()` de la clase `System.Console` ¿Cómo funciona el método `ReadKey()` de la misma clase? Escribir un programa que imprima en la consola la frase “Hola Mundo” haciendo una pausa entre palabra y palabra esperando a que el usuario presione una tecla para continuar.

Tip: usar los métodos `ReadKey()` y `Write()` de la clase `System.Console`

2) Investigar por las secuencias de escape `\n`, `\t`, `\"` y `\\`. Escribir un programa que las utilice para imprimir distintos mensajes en la consola.

3) El carácter `@` delante de un `string` desactiva los códigos de escape. Probar el siguiente fragmento de código, eliminar el carácter `@` y utilizar las secuencias de escape necesarias para que el programa siga funcionando de igual manera

```
string st = @"c:\windows\system";  
Console.WriteLine(st);
```

4) Escribir un programa que solicite al usuario ingresar su nombre e imprima en la consola un saludo personalizado utilizando ese nombre o la frase “Hola mundo” si el usuario ingresó una línea en blanco. Para ingresar un `string` desde el teclado utilizar `Console.ReadLine()`

5) Idem. al ejercicio anterior salvo que se imprimirá un mensaje de saludo diferente según sea el nombre ingresado por el usuario. Así para “Juan” debe imprimir “¡Hola amigo!”, para “María” debe imprimir “Buen día señora”, para “Alberto” debe imprimir “Hola Alberto”. En otro caso, debe imprimir “Buen día ” seguido del nombre ingresado o “¡Buen día mundo!” si se ha ingresado una línea vacía.

- a) utilizando `if ... else if`
- b) utilizando `switch`

6) Utilizar `Console.ReadLine()` para leer líneas de texto (secuencia de caracteres que finaliza al presionar <ENTER>) por la consola. Por cada línea leída se debe imprimir inmediatamente la cantidad de caracteres de la misma. El programa termina al ingresar la cadena vacía.
Tip: si `st` es una variable de tipo `string`, entonces `st.Length` devuelve la cantidad de caracteres del `string`.

7) Qué hace la instrucción `Console.WriteLine("100".Length);` ?

8) Sea `st` una variable de tipo `string` correctamente declarada. ¿Es válida la siguiente instrucción: `Console.WriteLine(st=Console.ReadLine());`?

9) Escribir un programa que lea dos palabras separadas por un blanco que terminan con <ENTER>, y determinar si son simétricas (Ej: 'abbccd' y 'dccbba' son simétricas).
Tip: si `st` es un `string`, entonces `st[0]` devuelve el primer carácter de `st`, y `st[st.Length-1]` devuelve el último carácter de `st`.

10) Escribir un programa que imprima en la consola todos los múltiplos de 17 o de 29 comprendidos entre 1 y 1000.

11) Comprobar el funcionamiento del siguiente fragmento de código, analizar el resultado y contestar las preguntas.

```
Console.WriteLine("10/3 = " + 10 / 3);  
Console.WriteLine("10.0/3 = " + 10.0 / 3);  
Console.WriteLine("10/3.0 = " + 10 / 3.0);  
int a = 10, b = 3;  
Console.WriteLine("Si a y b son variables enteras, si a=10 y b=3");  
Console.WriteLine("entonces a/b = " + a / b);  
double c = 3;  
Console.WriteLine("Si c es una variable double, c=3");  
Console.WriteLine("entonces a/c = " + a / c);
```

- a) ¿Qué se puede concluir respecto del operador de división “/” ?
- b) ¿Cómo funciona el operador + entre un **string** y un dato numérico?

12) Escribir un programa que imprima todos los divisores de un número entero ingresado desde la consola. Para obtener el entero desde un **string st** utilizar **int.Parse(st)**.

13) Si **a** y **b** son variables enteras, identificar el problema (y la forma de resolverlo) de la siguiente expresión. Tip: observar qué pasa cuando **b = 0**.

```
if ((b != 0) & (a/b > 5)) Console.WriteLine(a/b);
```

14) Utilizar el operador ternario condicional para establecer el contenido de una variable entera con el menor valor de otras dos variables enteras.

15) ¿Cuál es el problema del código siguiente y cómo se soluciona?

```
int i = 0;
for (int i = 1; i <= 10;)
{
    Console.WriteLine(i++);
}
```

16) Analizar el siguiente código. ¿Cuál es la salida por consola?

```
int i = 1;
if (--i == 0)
{
    Console.WriteLine("cero");
}
if (i++ == 0)
{
    Console.WriteLine("cero");
}
Console.WriteLine(i);
```